

INTERNATIONAL STANDARD

**Industrial communication networks – Fieldbus specifications –
Part 5-8: Application layer service definition – Type 8 elements**



THIS PUBLICATION IS COPYRIGHT PROTECTED

Copyright © 2007 IEC, Geneva, Switzerland

All rights reserved. Unless otherwise specified, no part of this publication may be reproduced or utilized in any form or by any means, electronic or mechanical, including photocopying and microfilm, without permission in writing from either IEC or IEC's member National Committee in the country of the requester.

If you have any questions about IEC copyright or have an enquiry about obtaining additional rights to this publication, please contact the address below or your local IEC member National Committee for further information.

IEC Central Office
3, rue de Varembe
CH-1211 Geneva 20
Switzerland
Email: inmail@iec.ch
Web: www.iec.ch

About the IEC

The International Electrotechnical Commission (IEC) is the leading global organization that prepares and publishes International Standards for all electrical, electronic and related technologies.

About IEC publications

The technical content of IEC publications is kept under constant review by the IEC. Please make sure that you have the latest edition, a corrigenda or an amendment might have been published.

- Catalogue of IEC publications: www.iec.ch/searchpub

The IEC on-line Catalogue enables you to search by a variety of criteria (reference number, text, technical committee,...). It also gives information on projects, withdrawn and replaced publications.

- IEC Just Published: www.iec.ch/online_news/justpub

Stay up to date on all new IEC publications. Just Published details twice a month all new publications released. Available on-line and also by email.

- Electropedia: www.electropedia.org

The world's leading online dictionary of electronic and electrical terms containing more than 20 000 terms and definitions in English and French, with equivalent terms in additional languages. Also known as the International Electrotechnical Vocabulary online.

- Customer Service Centre: www.iec.ch/webstore/custserv

If you wish to give us your feedback on this publication or need further assistance, please visit the Customer Service Centre FAQ or contact us:

Email: csc@iec.ch
Tel.: +41 22 919 02 11
Fax: +41 22 919 03 00



IEC 61158-5-8

Edition 1.0 2007-12

INTERNATIONAL STANDARD

**Industrial communication networks – Fieldbus specifications –
Part 5-8: Application layer service definition – Type 8 elements**

INTERNATIONAL
ELECTROTECHNICAL
COMMISSION

PRICE CODE

XD

ICS 35.100.70; 25.040.40

ISBN 2-8318-9456-5

LICENSED TO MECON Limited, - RANCHI/BANGALORE
FOR INTERNAL USE AT THIS LOCATION ONLY, SUPPLIED BY BOOK SUPPLY BUREAU.

CONTENTS

FOREWORD.....	4
INTRODUCTION.....	6
1 Scope.....	7
1.1 Overview.....	7
1.2 Specifications.....	8
1.3 Conformance.....	8
2 Normative references	8
3 Terms and definitions	9
3.1 ISO/IEC 7498-1 terms	9
3.2 ISO/IEC 8822 terms	9
3.3 ISO/IEC 9545 terms	9
3.4 ISO/IEC 8824 terms	9
3.5 Fieldbus data-link layer terms.....	9
3.6 Fieldbus application layer specific definitions	10
3.7 Abbreviations and symbols.....	16
3.8 Conventions	16
4 Concepts.....	20
4.1 Overview.....	20
4.2 Architectural relationships	20
4.3 Fieldbus Application Layer structure.....	22
4.4 FAL naming and addressing	32
4.5 Architecture summary.....	33
4.6 FAL service procedures.....	33
4.7 Notional FAL service procedures [INFORMATIVE].....	33
4.8 Common FAL attributes.....	34
4.9 Common FAL service parameters.....	35
4.10 APDU size.....	36
5 Data type ASE.....	36
5.1 General.....	36
5.2 Formal definition of data type objects	38
5.3 FAL defined data types.....	38
5.4 Data type ASE service specification	42
6 Communication model specification	42
6.1 ASEs.....	42
6.2 ARs.....	85
6.3 Summary of FAL classes	91
6.4 Permitted FAL services by AREP role.....	91
Bibliography.....	93
Figure 1 – Relationship to the OSI basic reference model.....	20
Figure 2 – Architectural positioning of the Type 8 FAL	21
Figure 3 – Client/server interactions	23
Figure 4 – Push model interactions	24
Figure 5 – APOs services conveyed by the FAL.....	25
Figure 6 – Application entity structure	27

Figure 7 – Example FAL ASEs	28
Figure 8 – FAL management of objects	29
Figure 9 – ASE service conveyance	30
Figure 10 – Defined and established AREPs	32
Figure 11 – FAL architectural components	33
Figure 12 – Data type class hierarchy example	36
Figure 13 – The AR ASE conveys APDUs between APs	58
Figure 14 – 1-to-1 AR establishment	65
Table 1 – Get attributes service parameters	43
Table 2 – Identify	50
Table 3 – Get status	51
Table 4 – Initiate	53
Table 5 – Terminate	55
Table 6 – Reject	57
Table 7 – Conveyance of service primitives by AREP role	59
Table 8 – Valid combinations of AREP roles involved in an AR	59
Table 9 – AR-unconfirmed send	63
Table 10 – AR-establish service	64
Table 11 – Valid combinations of AREP classes to be related	66
Table 12 – AR-abort	67
Table 13 – AR-Data-Send-Acknowledge service parameters	68
Table 14 – Read service parameters	73
Table 15 – Write service parameters	74
Table 16 – Information report service	75
Table 17 – Start service parameters	79
Table 18 – Stop service parameters	80
Table 19 – Resume service parameters	81
Table 20 – Reset service parameters	82
Table 21 – State transitions for a function invocation object	84
Table 22 – FAL class summary	91
Table 23 – Services by AREP role	92

INTERNATIONAL ELECTROTECHNICAL COMMISSION

**INDUSTRIAL COMMUNICATION NETWORKS –
FIELD BUS SPECIFICATIONS –****Part 5-8: Application layer service definition – Type 8 elements**

FOREWORD

- 1) The International Electrotechnical Commission (IEC) is a worldwide organization for standardization comprising all national electrotechnical committees (IEC National Committees). The object of IEC is to promote international co-operation on all questions concerning standardization in the electrical and electronic fields. To this end and in addition to other activities, IEC publishes International Standards, Technical Specifications, Technical Reports, Publicly Available Specifications (PAS) and Guides (hereafter referred to as "IEC Publication(s)"). Their preparation is entrusted to technical committees; any IEC National Committee interested in the subject dealt with may participate in this preparatory work. International, governmental and non-governmental organizations liaising with the IEC also participate in this preparation. IEC collaborates closely with the International Organization for Standardization (ISO) in accordance with conditions determined by agreement between the two organizations.
- 2) The formal decisions or agreements of IEC on technical matters express, as nearly as possible, an international consensus of opinion on the relevant subjects since each technical committee has representation from all interested IEC National Committees.
- 3) IEC Publications have the form of recommendations for international use and are accepted by IEC National Committees in that sense. While all reasonable efforts are made to ensure that the technical content of IEC Publications is accurate, IEC cannot be held responsible for the way in which they are used or for any misinterpretation by any end user.
- 4) In order to promote international uniformity, IEC National Committees undertake to apply IEC Publications transparently to the maximum extent possible in their national and regional publications. Any divergence between any IEC Publication and the corresponding national or regional publication shall be clearly indicated in the latter.
- 5) IEC provides no marking procedure to indicate its approval and cannot be rendered responsible for any equipment declared to be in conformity with an IEC Publication.
- 6) All users should ensure that they have the latest edition of this publication.
- 7) No liability shall attach to IEC or its directors, employees, servants or agents including individual experts and members of its technical committees and IEC National Committees for any personal injury, property damage or other damage of any nature whatsoever, whether direct or indirect, or for costs (including legal fees) and expenses arising out of the publication, use of, or reliance upon, this IEC Publication or any other IEC Publications.
- 8) Attention is drawn to the Normative references cited in this publication. Use of the referenced publications is indispensable for the correct application of this publication.
- 9) Attention is drawn to the possibility that some of the elements of this IEC Publication may be the subject of patent rights. IEC shall not be held responsible for identifying any or all such patent rights.

NOTE Use of some of the associated protocol types is restricted by their intellectual-property-right holders. In all cases, the commitment to limited release of intellectual-property-rights made by the holders of those rights permits a particular data-link layer protocol type to be used with physical layer and application layer protocols in Type combinations as specified explicitly in the IEC 61784 series. Use of the various protocol types in other combinations may require permission from their respective intellectual-property-right holders.

International Standard IEC 61158-5-8 has been prepared by subcommittee 65C: Industrial networks, of IEC technical committee 65: Industrial-process measurement, control and automation.

This first edition and its companion parts of the IEC 61158-5 subseries cancel and replace IEC 61158-5:2003. This edition of this part constitutes a technical revision.

This edition of IEC 61158-5 includes the following significant changes from the previous edition:

- a) deletion of the former Type 6 fieldbus for lack of market relevance;
- b) addition of new types of fieldbuses;
- c) partition of part 5 of the third edition into multiple parts numbered -5-2, -5-3, ...

The text of this standard is based on the following documents:

FDIS	Report on voting
65C/475/FDIS	65C/486/RVD

Full information on the voting for the approval of this standard can be found in the report on voting indicated in the above table.

This publication has been drafted in accordance with ISO/IEC Directives, Part 2.

The committee has decided that the contents of this publication will remain unchanged until the maintenance result date indicated on the IEC web site under <http://webstore.iec.ch> in the data related to the specific publication. At this date, the publication will be:

- reconfirmed;
- withdrawn;
- replaced by a revised edition, or
- amended.

NOTE The revision of this standard will be synchronized with the other parts of the IEC 61158 series.

The list of all the parts of the IEC 61158 series, under the general title *Industrial communication networks – Fieldbus specifications*, can be found on the IEC web site.

INTRODUCTION

This part of IEC 61158 is one of a series produced to facilitate the interconnection of automation system components. It is related to other standards in the set as defined by the “three-layer” fieldbus reference model described in IEC/TR 61158-1.

The application service is provided by the application protocol making use of the services available from the data-link or other immediately lower layer. This standard defines the application service characteristics that fieldbus applications and/or system management may exploit.

Throughout the set of fieldbus standards, the term “service” refers to the abstract capability provided by one layer of the OSI Basic Reference Model to the layer immediately above. Thus, the application layer service defined in this standard is a conceptual architectural service, independent of administrative and implementation divisions.

INDUSTRIAL COMMUNICATION NETWORKS – FIELDBUS SPECIFICATIONS –

Part 5-8: Application layer service definition – Type 8 elements

1 Scope

1.1 Overview

The fieldbus application layer (FAL) provides user programs with a means to access the fieldbus communication environment. In this respect, the FAL can be viewed as a “window between corresponding application programs.”

This standard provides common elements for basic time-critical and non-time-critical messaging communications between application programs in an automation environment and material specific to Type 8 fieldbus. The term “time-critical” is used to represent the presence of a time-window, within which one or more specified actions are required to be completed with some defined level of certainty. Failure to complete specified actions within the time window risks failure of the applications requesting the actions, with attendant risk to equipment, plant and possibly human life.

This standard defines in an abstract way the externally visible service provided by the Type 8 fieldbus application layer in terms of

- a) an abstract model for defining application resources (objects) capable of being manipulated by users via the use of the FAL service,
- b) the primitive actions and events of the service;
- c) the parameters associated with each primitive action and event, and the form which they take; and
- d) the interrelationship between these actions and events, and their valid sequences.

The purpose of this standard is to define the services provided to

- 1) the FAL user at the boundary between the user and the application layer of the fieldbus reference model, and
- 2) Systems Management at the boundary between the application layer and Systems Management of the fieldbus reference model.

This standard specifies the structure and services of the Type 8 fieldbus application layer, in conformance with the OSI Basic Reference Model (ISO/IEC 7498) and the OSI application layer structure (ISO/IEC 9545).

FAL services and protocols are provided by FAL application-entities (AE) contained within the application processes. The FAL AE is composed of a set of object-oriented application service elements (ASEs) and a layer management entity (LME) that manages the AE. The ASEs provide communication services that operate on a set of related application process object (APO) classes. One of the FAL ASEs is a management ASE that provides a common set of services for the management of the instances of FAL classes.

Although these services specify, from the perspective of applications, how request and responses are issued and delivered, they do not include a specification of what the requesting and responding applications are to do with them. That is, the behavioral aspects of the applications are not specified; only a definition of what requests and responses they can send/receive is specified. This permits greater flexibility to the FAL users in standardizing

such object behavior. In addition to these services, some supporting services are also defined in this standard to provide access to the FAL to control certain aspects of its operation.

1.2 Specifications

The principal objective of this standard is to specify the characteristics of conceptual application layer services suitable for time-critical communications, and thus supplement the OSI Basic Reference Model in guiding the development of application layer protocols for time-critical communications.

A secondary objective is to provide migration paths from previously-existing industrial communications protocols. It is this latter objective which gives rise to the diversity of services standardized as the various Types of IEC 61158, and the corresponding protocols standardized in subparts of IEC 61158-6.

This specification may be used as the basis for formal application programming interfaces. Nevertheless, it is not a formal programming interface, and any such interface will need to address implementation issues not covered by this specification, including

- a) the sizes and octet ordering of various multi-octet service parameters, and
- b) the correlation of paired request and confirm, or indication and response, primitives.

1.3 Conformance

This standard does not specify individual implementations or products, nor does it constrain the implementations of application layer entities within industrial automation systems.

There is no conformance of equipment to this application layer service definition standard. Instead, conformance is achieved through implementation of conforming application layer protocols that fulfill the Type 8 application layer services as defined in this standard.

2 Normative references

The following referenced documents are indispensable for the application of this document. For dated references, only the edition cited applies. For undated references, the latest edition of the referenced document (including any amendments) applies.

IEC 60559, *Binary Floating-point Arithmetic for Microprocessor Systems*

ISO/IEC 646, *Information technology – ISO 7-bit coded character set for information interchange*

ISO/IEC 7498–1, *Information technology – Open Systems Interconnection – Basic Reference Model – Part 1: The Basic Model*

ISO/IEC 7498–3, *Information technology – Open Systems Interconnection – Basic Reference Model – Part 3: Naming and addressing*

ISO/IEC 8822, *Information technology – Open Systems Interconnection – Presentation service definition*

ISO/IEC 8824, *Information Technology – Abstract Syntax notation One (ASN-1): Specification of basic notation*

ISO/IEC 9545, *Information technology – Open Systems Interconnection – Application Layer structure*

ISO/IEC 10731, *Information technology – Open Systems Interconnection – Basic Reference Model – Conventions for the definition of OSI services*

3 Terms and definitions

For the purposes of this document, the following terms, definitions, symbols, abbreviations and conventions apply.

3.1 ISO/IEC 7498-1 terms

This standard is partly based on the concepts developed in ISO/IEC 7498-1, and makes use of the following terms defined therein:

- a) application entity
- b) application process
- c) application protocol data unit
- d) application process invocation
- e) application transaction
- f) real open system
- g) transfer syntax

3.2 ISO/IEC 8822 terms

For the purposes of this document, the following terms as defined in ISO/IEC 8822 apply:

- a) abstract syntax
- b) presentation context

3.3 ISO/IEC 9545 terms

For the purposes of this document, the following terms as defined in ISO/IEC 9545 apply:

- a) application-association
- b) application-context
- c) application context name
- d) application-entity-invocation
- e) application-entity-type
- f) application-process-invocation
- g) application-process-type
- h) application-service-element
- i) application control service element

3.4 ISO/IEC 8824 terms

For the purposes of this document, the following terms as defined in ISO/IEC 8824 apply:

- a) object identifier
- b) type

3.5 Fieldbus data-link layer terms

For the purposes of this document, the following terms, as defined in IEC 61158-3-8 and IEC 61158-4-8, apply.

- a) DLCEP

- b) DLSAP
- c) link
- d) node

3.6 Fieldbus application layer specific definitions

3.6.1

access protection

limitation of the usage of an application object to one client

3.6.2

allocate

take a resource from a common area and assign that resource for the exclusive use of a specific entity

3.6.3

application

function or data structure for which data is consumed or produced

3.6.4

application layer interoperability

capability of application entities to perform coordinated and cooperative operations using the services of the FAL

3.6.5

application objects

multiple object classes that manage and provide a run time exchange of messages across the network and within the network device

3.6.6

application process

part of a distributed application on a network, which is located on one device and unambiguously addressed

3.6.7

application process identifier

distinguishes multiple application processes used in a device

3.6.8

application process object

component of an application process that is identifiable and accessible through an FAL application relationship

NOTE Application process object definitions are composed of a set of values for the attributes of their class (see the definition for Application Process Object Class Definition). Application process object definitions may be accessed remotely using the services of the FAL Object Management ASE. FAL Object Management services can be used to load or update object definitions, to read object definitions, and to dynamically create and delete application objects and their corresponding definitions.

3.6.9

application process object class

a class of application process objects defined in terms of the set of their network-accessible attributes and services

3.6.10

application relationship

cooperative association between two or more application-entity-invocations for the purpose of exchange of information and coordination of their joint operation. This relationship is activated

either by the exchange of application-protocol-data-units or as a result of preconfiguration activities

3.6.11

application relationship application service element

application-service-element that provides the exclusive means for establishing and terminating all application relationships

3.6.12

application relationship endpoint

context and behavior of an application relationship as seen and maintained by one of the application processes involved in the application relationship

NOTE Each application process involved in the application relationship maintains its own application relationship endpoint.

3.6.13

attribute

description of an externally visible characteristic or feature of an object

NOTE The attributes of an object contain information about variable portions of an object. Typically, they provide status information or govern the operation of an object. Attributes may also affect the behaviour of an object. Attributes are divided into class attributes and instance attributes.

3.6.14

behaviour

indication of how an object responds to particular eventss

3.6.15

class

a set of objects, all of which represent the same kind of system component

NOTE A class is a generalisation of an object; a template for defining variables and methods. All objects in a class are identical in form and behaviour, but usually contain different data in their attributes.

3.6.16

class attribute

attribute that is shared by all objects within the same class

3.6.17

class code

unique identifier assigned to each object class

3.6.18

class specific service

service defined by a particular object class to perform a required function which is not performed by a common service

NOTE A class specific object is unique to the object class which defines it.

3.6.19

client

- a) object which uses the services of another (server) object to perform a task
- b) initiator of a message to which a server reacts

3.6.20

control commands

action invocations transferred from client to server to clear outputs, freeze inputs and/or synchronise outputs

3.6.21

conveyance path

unidirectional flow of APDUs across an application relationship

3.6.22

cyclic

repetitive in a regular manner

3.6.23

data consistency

means for coherent transmission and access of the input- or output-data object between and within client and server

3.6.24

dedicated AR

AR used directly by the FAL User

NOTE On Dedicated ARs, only the FAL Header and the user data are transferred.

3.6.25

device

physical hardware connected to the link

NOTE A device may contain more than one node.

3.6.26

dynamic AR

AR that requires the use of the AR establishment procedures to place it into an established state

3.6.27

end node

producing or consuming node

3.6.28

endpoint

one of the communicating entities involved in a connection

3.6.29

engineering

abstract term that characterizes the client application or device responsible for configuring an automation system via interconnecting data items

3.6.30

error

discrepancy between a computed, observed or measured value or condition and the specified or theoretically correct value or condition

3.6.31

error class

general grouping for related error definitions and corresponding error codes

3.6.32

error code

identification of a specific type of error within an error class

3.6.33

event

an instance of a change of conditions

3.6.34**group**

- (a) collection of objects
- (b) address that identifies more than one entity

3.6.35**interface**

- a) shared boundary between two functional units, defined by functional characteristics, signal characteristics, or other characteristics as appropriate
- b) collection of FAL class attributes and services that represents a specific view on the FAL class

3.6.36**invocation**

act of using a service or other resource of an application process

NOTE Each invocation represents a separate thread of control that may be described by its context. Once the service completes, or use of the resource is released, the invocation ceases to exist. For service invocations, a service that has been initiated but not yet completed is referred to as an outstanding service invocation.

3.6.37**index**

address of an object within an application process

3.6.38**instance**

the actual physical occurrence of an object within a class that identifies one of many objects within the same object class

NOTE The terms object, instance, and object instance are used to refer to a specific instance.

3.6.39**instance attributes**

attribute that is unique to an object instance and not shared by the object class

3.6.40**instantiated**

object that has been created in a device

3.6.41**manufacturer ID**

identification of each product manufacturer by a unique number

3.6.42**management information**

network-accessible information that supports managing the operation of the fieldbus system, including the application layer

NOTE Managing includes functions such as controlling, monitoring, and diagnosing.

3.6.43**member**

piece of an attribute that is structured as an element of an array

3.6.44**method**

<object> a synonym for an operational service which is provided by the server ASE and invoked by a client

3.6.45

network

a set of nodes connected by some type of communication medium, including any intervening repeaters, bridges, routers and lower-layer gateways

3.6.46

object

abstract representation of a particular component within a device, usually a collection of related data (in the form of variables) and methods (procedures) for operating on that data that have clearly defined interface and behaviour

3.6.47

object specific service

service unique to the object class which defines it

3.6.48

peer

role of an AR endpoint in which it is capable of acting as both client and server

3.6.49

point-to-point connection

connection that exists between exactly two application objects

3.6.50

pre-defined AR endpoint

AR endpoint that is defined locally within a device without use of the create service

NOTE Pre-defined ARs that are not pre-established are established before being used

3.6.51

pre-established AR endpoint

AR endpoint that is placed in an established state during configuration of the AEs that control its endpoints

3.6.52

property

descriptive information about an object

3.6.53

provider

source of a data connection

3.6.54

publisher

role of an AR endpoint that transmits APDUs onto the fieldbus for consumption by one or more subscribers

NOTE A publisher may not be aware of the identity or the number of subscribers and it may publish its APDUs using a dedicated AR.

3.6.55

push publisher

type of publisher that publishes an object in an unconfirmed service request APDU

3.6.56

push subscriber

type of subscriber that recognizes received unconfirmed service request APDUs as published object data

3.6.57

resource

a processing or information capability of a subsystem

3.6.58

server

- a) role of an AREP in which it returns a confirmed service response APDU to the client that initiated the request
- b) object which provides services to another (client) object

3.6.59

service

operation or function than an object and/or object class performs upon request from another object and/or object class

3.6.60

subscriber

role of an AREP in which it receives APDUs produced by a publisher

3.7 Abbreviations and symbols

AE	Application entity
AL	Application layer
ALME	Application layer management entity
ALP	Application layer protocol
APO	Application object
AP	Application process
APDU	Application protocol data unit
API	Application process identifier
AR	Application relationship
AREP	Application relationship endpoint
ASE	Application service element
CIM	Computer integrated manufacturing
Cnf	Confirmation
CR	Communication relationship
CREP	Communication relationship endpoint
DL-	(as a prefix) Data-link-
DLC	Data-link connection
DLCEP	Data-link connection endpoint
DLL	Data-link layer
DLM	Data-link-management
DLSAP	Data-link service access point
FAL	Fieldbus application layer
FIFO	First in first out
ID	Identifier
Ind	Indication
LME	Layer management entity
OSI	Open Systems Interconnect
PDU	Protocol data unit
Req	Request
Rsp	Response
SAP	Service access point

3.8 Conventions

3.8.1 Overview

The FAL is defined as a set of object-oriented ASEs. Each ASE is specified in a separate subclause. Each ASE specification is composed of two parts, its class specification, and its service specification.

The class specification defines the attributes of the class. The attributes are accessible from instances of the class using the Object Management ASE services specified in Clause 5 of this standard. The service specification defines the services that are provided by the ASE.

3.8.2 General conventions

This standard uses the descriptive conventions given in ISO/IEC 10731.

3.8.3 Conventions for class definitions

Class definitions are described using templates. Each template consists of a list of attributes for the class. The general form of the template is shown below:

FAL ASE:		ASE Name
CLASS:		Class Name
CLASS ID:		#
PARENT CLASS:		Parent Class Name
ATTRIBUTES:		
1	(o)	Key Attribute: numeric identifier
2	(o)	Key Attribute: name
3	(m)	Attribute: attribute name(values)
4	(m)	Attribute: attribute name(values)
4.1	(s)	Attribute: attribute name(values)
4.2	(s)	Attribute: attribute name(values)
4.3	(s)	Attribute: attribute name(values)
5.	(c)	Constraint: constraint expression
5.1	(m)	Attribute: attribute name(values)
5.2	(o)	Attribute: attribute name(values)
6	(m)	Attribute: attribute name(values)
6.1	(s)	Attribute: attribute name(values)
6.2	(s)	Attribute: attribute name(values)
SERVICES:		
1	(o)	OpsService: service name
2.	(c)	Constraint: constraint expression
2.1	(o)	OpsService: service name
3	(m)	MgtService: service name

- (1) The "FAL ASE:" entry is the name of the FAL ASE that provides the services for the class being specified.
- (2) The "CLASS:" entry is the name of the class being specified. All objects defined using this template will be an instance of this class. The class may be specified by this standard, or by a user of this standard.
- (3) The "CLASS ID:" entry is a number that identifies the class being specified. This number is unique within the FAL ASE that will provide the services for this class. When qualified by the identity of its FAL ASE, it unambiguously identifies the class within the scope of the FAL. The value "NULL" indicates that the class cannot be instantiated. Class IDs between 1 and 255 are reserved by this standard to identify standardized classes. They have been assigned to maintain compatibility with existing national standards. CLASS IDs between 256 and 2048 are allocated for identifying user defined classes.
- (4) The "PARENT CLASS:" entry is the name of the parent class for the class being specified. All attributes defined for the parent class and inherited by it are inherited for the class being defined, and therefore do not have to be redefined in the template for this class.

NOTE The parent-class "TOP" indicates that the class being defined is an initial class definition. The parent class TOP is used as a starting point from which all other classes are defined. The use of TOP is reserved for classes defined by this standard.

- (5) The "ATTRIBUTES" label indicate that the following entries are attributes defined for the class.
- a) Each of the attribute entries contains a line number in column 1, a mandatory (m) / optional (o) / conditional (c) / selector (s) indicator in column 2, an attribute type label in column 3, a name or a conditional expression in column 4, and optionally a list of enumerated values in column 5. In the column following the list of values, the default value for the attribute may be specified.
 - b) Objects are normally identified by a numeric identifier or by an object name, or by both. In the class templates, these key attributes are defined under the key attribute.
 - c) The line number defines the sequence and the level of nesting of the line. Each nesting level is identified by period. Nesting is used to specify
 - i) fields of a structured attribute (4.1, 4.2, 4.3),
 - ii) attributes conditional on a constraint statement (5). Attributes may be mandatory (5.1) or optional (5.2) if the constraint is true. Not all optional attributes require constraint statements as does the attribute defined in (5.2).
 - iii) the selection fields of a choice type attribute (6.1 and 6.2).
- (6) The "SERVICES" label indicates that the following entries are services defined for the class.
- a) An (m) in column 2 indicates that the service is mandatory for the class, while an (o) indicates that it is optional. A (c) in this column indicates that the service is conditional. When all services defined for a class are defined as optional, at least one has to be selected when an instance of the class is defined.
 - b) The label "OpsService" designates an operational service (1).
 - c) The label "MgtService" designates an management service (2).
 - d) The line number defines the sequence and the level of nesting of the line. Each nesting level is identified by period. Nesting within the list of services is used to specify services conditional on a constraint statement.

3.8.4 Conventions for service definitions

3.8.4.1 General

The service model, service primitives, and time-sequence diagrams used are entirely abstract descriptions; they do not represent a specification for implementation.

3.8.4.2 Service parameters

Service primitives are used to represent service user/service provider interactions (ISO/IEC 10731). They convey parameters which indicate information available in the user/provider interaction. In any particular interface, not all parameters need be explicitly stated.

The service specifications of this standard uses a tabular format to describe the component parameters of the ASE service primitives. The parameters which apply to each group of service primitives are set out in tables. Each table consists of up to five columns for the

- 1) parameter name,
- 2) request primitive,
- 3) indication primitive,
- 4) response primitive, and
- 5) confirm primitive.

One parameter (or component of it) is listed in each row of each table. Under the appropriate service primitive columns, a code is used to specify the type of usage of the parameter on the primitive specified in the column:

- M parameter is mandatory for the primitive
- U parameter is a User option, and may or may not be provided depending on dynamic usage of the service user. When not provided, a default value for the parameter is assumed.
- C parameter is conditional upon other parameters or upon the environment of the service user.
- (blank) parameter is never present.
- S parameter is a selected item.

Some entries are further qualified by items in brackets. These may be

- a) a parameter-specific constraint:
“(=)” indicates that the parameter is semantically equivalent to the parameter in the service primitive to its immediate left in the table.
- b) an indication that some note applies to the entry:
“(n)” indicates that the following note "n" contains additional information pertaining to the parameter and its use.

3.8.4.3 Service procedures

The procedures are defined in terms of

- the interactions between application entities through the exchange of fieldbus Application Protocol Data Units, and
- the interactions between an application layer service provider and an application layer service user in the same system through the invocation of application layer service primitives.

These procedures are applicable to instances of communication between systems which support time-constrained communications services within the fieldbus Application Layer.

4 Concepts

4.1 Overview

This subclause describes fundamentals of the FAL. Detailed descriptive information about each of the FAL ASEs can be found in the “Overview” subclause of each of the Communication Model specifications.

4.2 Architectural relationships

4.2.1 Relationship to the Application Layer of the OSI basic reference model

The functions of the FAL have been described according to OSI layering principles. However, its architectural relationship to the lower layers is different, as shown in Figure 1.

- The FAL includes OSI functions together with extensions to cover time-critical requirements. The OSI Application Layer Structure standard (ISO/IEC 9545) was used as a basis for specifying the FAL.
- The FAL directly uses the services of the underlying layer. The underlying layer may be the data link layer or any layer in between. When using the underlying layer, the FAL may provide functions normally associated with the OSI Middle Layers for proper mapping onto the underlying layer.

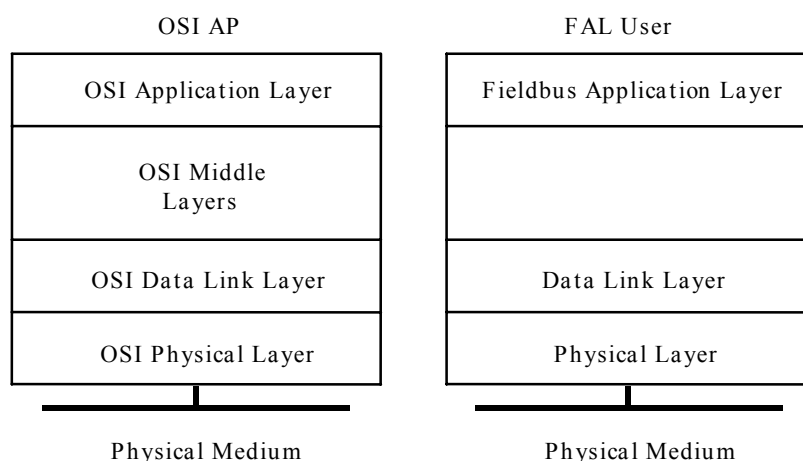


Figure 1 – Relationship to the OSI basic reference model

4.2.2 Relationships to other fieldbus entities

4.2.2.1 General

The Type 8 Fieldbus Application Layer (FAL) architectural relationships, as illustrated in Figure 2, have been designed to support the interoperability needs of time-critical systems distributed within the fieldbus environment.

Within this environment, the FAL provides communications services to time-critical and non-time-critical applications located in Type 8 devices.

In addition, the FAL directly uses the data-link layer to transfer its application layer protocol data units. It does this using a set of data transfer services and a set of supporting services used to control the operational aspects of the data-link layer.

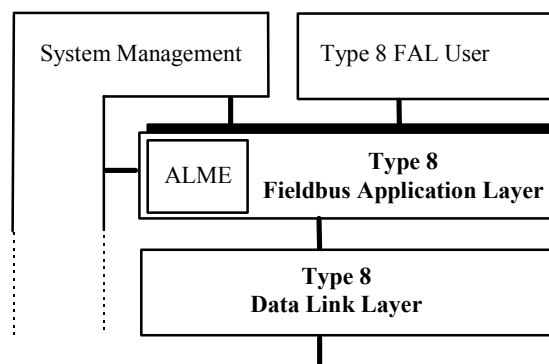


Figure 2 – Architectural positioning of the Type 8 FAL

4.2.3 Mapping of functional capabilities

Two communication mechanisms are supported:

- cyclic transmission of data in a high efficient manner using a publisher subscriber model,
- non cyclic transmission of data using a client server communication model

AREPs, which act as push publisher or push subscriber, are used for cyclic transmission.

AREPs, which act as client and server (peer), are used for non cyclic data transmission. A specific transfer syntax is used for their APDUs. The non cyclic data transmission mechanism is described by a subset of those ASEs and services specified in 6.1.

4.2.3.1 Use of the Type 8 data-link layer

The fieldbus Application Layer (FAL) provides network access to fieldbus APs. It interfaces directly to the fieldbus data-link layer for transfer of its APDUs.

The data-link layer provides various types of services to the FAL for the transfer of data between Data-link endpoints (e.g. DLSAPs, DLCEPs).

4.2.3.2 Support for Type 8 applications

TYPE 8 applications are represented to the network as application processes (APs). APs are the components of a distributed system that may be individually identified and addressed.

Each AP contains an FAL application entity (AE) that provides network access for the AP. That is, each AP communicates with other APs through its AE. In this sense, the AE provides a window of visibility into the AP.

APs contain identifiable components that are also visible across the network. These components are represented to the network as Application Process Objects (APO). They may be identified by one or more key attributes. They are located at the address of the application process that contains them.

The services used to access them are provided by APO-specific application service elements (ASEs) contained within the FAL. These ASEs are designed to support user, function block, and management applications.

4.2.3.3 Support for system management

The FAL services can be used to support various management operations, including management of fieldbus systems, applications, and the fieldbus network.

4.2.3.4 Access to FAL layer management entities

One layer management entity (LME) may be present in each FAL entity on the network. FALMEs provide access to the FAL for system management purposes.

The set of data accessible by the System Manager is referred to as the System Management Information Base (SMIB). Each fieldbus Application Layer Management Entity (FALME) provides the FAL portion of the SMIB. How the SMIB is implemented is beyond the scope of this standard.

4.3 Fieldbus Application Layer structure

4.3.1 Overview

The structure of the FAL is a refinement of the OSI Application Layer Structure (ISO/IEC 9545). As a result, the organization of this subclause is similar to that of ISO/IEC 9545. Certain concepts presented here have been refined from ISO/IEC 9545 for the TYPE 8 environment.

The FAL differs from the other layers of OSI in two principal aspects:

- OSI defines a single type of application layer communications channel, the association, to connect APs to each other. The FAL defines the Application Relationship (AR), of which there are several types, to permit application processes (APs) to communicate with each other.
- The FAL uses the DLL to transfer its PDUs and not the presentation layer. Therefore, there is no explicit presentation context available to the FAL. Between the same pair (or set) of data link service access points the FAL protocol may not be used concurrently with other application layer protocols.

4.3.2 Fundamental concepts

The operation of time critical real open systems is modeled in terms of interactions between time-critical APs. The FAL permits these APs to pass commands and data between them.

Cooperation between APs requires that they share sufficient information to interact and carry out processing activities in a coordinated manner. Their activities may be restricted to a single fieldbus segment, or they may span multiple segments. The FAL has been designed using a modular architecture to support the messaging requirements of these applications.

Cooperation between APs also sometimes requires that they share a common sense of time. The FAL or the data-link layer (part 3 and part 4) may provide for the distribution of time to all devices. They also may define local device services that can be used by APs to access the distributed time.

The remainder of this subclause describes each of the modular components of the architecture and their relationships with each other. The components of the FAL are modeled as objects, each of which provides a set of FAL communication services for use by applications. The FAL objects and their relationships are described below. The detailed specifications of FAL objects and their services are provided in the following clauses of this standard. Part 6 specifies the protocols necessary to convey these object services between applications.

4.3.3 Application Processes

4.3.3.1 Definition of the AP

In the TYPE 8 environment, an application may be partitioned into a set of components and distributed across a number of devices on the network. Each of these components is referred to as a fieldbus Application Process (AP). A fieldbus AP is a variation of an Application

Process as defined in ISO OSI Reference Model (ISO/IEC 7498). Fieldbus APs may be unambiguously addressed by at least one individual data-link layer service access point address. Unambiguously addressed, in this context, means that no other AP may simultaneously be located by the same address. This definition does not prohibit an AP from being located by more than one individual or group data link service access point address.

4.3.3.2 Communication services

APs communicate with each other using confirmed and unconfirmed services (ISO/IEC 10731). The services defined in this standard for the FAL specify the semantics of the services as seen by the requesting and responding APs. The syntax of the messages used to convey the service requests and responses is defined in part 6. The AP behavior associated with the services is specified by the AP.

Confirmed services are used to define request/response exchanges between APs.

Unconfirmed services, in contrast, are used to define the unidirectional transfer of messages from one AP to one or more remote APs. From a communications perspective, there is no relationship between separate invocations of unconfirmed services as there is between the request and response of a confirmed service.

4.3.3.3 AP interactions

4.3.3.3.1 General

APs may interact with other APs as necessary to achieve their functional objectives. No constraints are imposed by this standard on the organization of these interactions or the possible relationships that may exist between them.

For example, interactions may be based on request/response messages sent directly between APs, or on data/events sent by one AP for use by others. These two models of interactions between APs are referred to as client/server and publisher/subscriber interactions.

The services supported by an interaction model are conveyed by application relationship endpoints (AREPs) associated with the communicating APs. The role that the AREP plays in the interaction (e.g. client, server, peer, publisher, subscriber) is defined as an attribute of the AREP.

4.3.3.3.2 Client/server interactions

Client/server interactions are characterized by a bi-directional data flow between a client AP and one or more server APs. Figure 3 illustrates the interaction between a single client and a single server. In this type of interaction, the client may issue a confirmed or unconfirmed request to the server to perform some task. If the service is confirmed then the server will always return a response. If the service is unconfirmed, the server may return a response using an unconfirmed service defined for this purpose.

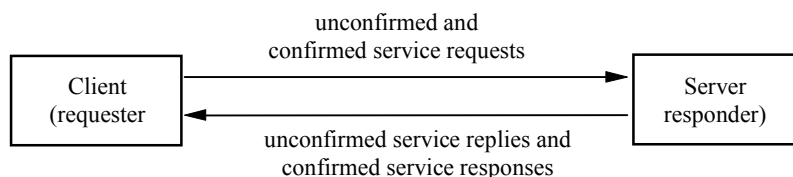


Figure 3 – Client/server interactions

4.3.3.3.3 Publisher/subscriber interactions

4.3.3.3.3.1 General

Publisher/subscriber interactions, on the other hand, involve a single publisher AP, and one subscriber APs. This type of interaction has been defined to support one model of interaction between APs, the "push" model. In the model, the setup of the publishing AP is performed by management and is outside the scope of this standard.

4.3.3.3.3.2 Push model interactions

In the "push" model, two services may be used, one confirmed and one unconfirmed. The confirmed service is used by the subscriber to request to join the publishing. The response to this request is returned to the subscriber, following the client/server model of interaction. This exchange is only necessary when the subscriber and the publisher are located in different APs.

The unconfirmed service used in the Push Model is used by the publisher to distribute its information to subscribers. In this case, the publisher is responsible for invoking the correct unconfirmed service at the appropriate time and for supplying the appropriate information. In this fashion, it is configured to "push" its data onto the network.

Subscribers for the Push Model receive the published unconfirmed services distributed by publishers. Figure 4 illustrates the concept of the Push Model.

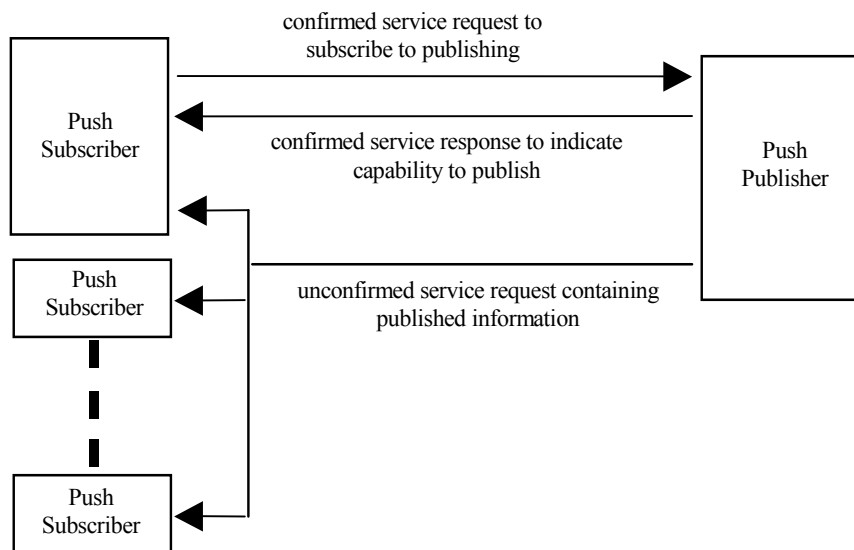


Figure 4 – Push model interactions

4.3.3.4 AP structure

The internals of APs may be represented by one, or more Application Process Objects (APOs) and accessed through one or more Application Entities (AEs). AEs provide the communication capabilities of the AP. For each AP, there is one and only one FAL AE. APOs are the network representation of application specific capabilities (user application process objects) of an AP that are accessible through its FAL AE.

4.3.3.5 AP class

An AP class is a definition of the attributes and services of an AP. The standard class definition for APs is defined in this standard. User defined classes also may be specified. Class identifiers (described in 3.8.2) are assigned from a set reserved for this purpose.

4.3.3.6 AP type

As described above in the previous subclauses, APs are defined by instantiating an AP class. Each AP definition is composed of the attributes and services selected for the AP from those defined by its AP class. In addition, an AP definition contains values for one or more of the attributes selected for it. When two APs share the same definition, that definition is referred to as an AP type. Thus, an AP type is a generic specification of an AP that may be used to define one or more APs.

4.3.4 Application process objects

4.3.4.1 Definition of APO

An application process object (APO) is a network representation of a specific aspect of an AP. Each APO represents a specific set of information and processing capabilities of an AP that are accessible through services of the FAL. APOs are used to represent these capabilities to other APs in a fieldbus system.

From the perspective of the FAL, an APO is modeled as a network accessible object contained within an AP or within another APO (APOs may contain other APOs). APOs provide the network definition for objects contained within an AP that are remotely accessible. The definition of an APO includes an identification of the FAL services that can be used by remote APs for remote access. The FAL services, as shown in Figure 5, are provided by the FAL communications entity of the AP, known as the FAL Applications Entity (FAL AE).

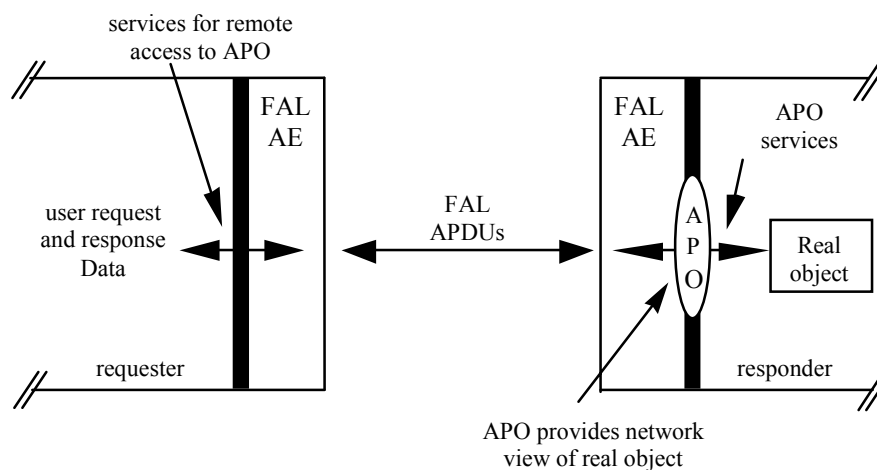


Figure 5 – APOs services conveyed by the FAL

In Figure 5, remote APs acting as clients may access the real object by sending requests through the APO that represents the real object. Local aspects of the AP convert between the network view (the APO) of the real object and the internal AP view of the real object.

To support the publisher/subscriber model of interaction, information about the real object can be published through its APO. Remote APs acting as subscribers see the APO view of the published information instead of having to know any of the real object specific details.

4.3.4.2 APO classes

An APO Class is a generic specification for a set of APOs, each of which is described by the same set of attributes and accessed using the same set of services.

APO Classes provide the mechanism for standardizing network visible aspects of APs. Each standard APO Class definition specifies a particular set of network accessible AP attributes and services. IEC 61158-6-8 specifies the syntax and the procedures used by the FAL protocol to provide remote access to the attributes and services of an APO Class.

Standard APO Classes are specified by this standard for the purpose of standardizing remote access to APs. User defined classes may also be specified.

User defined classes are defined as subclasses of standardized APO Classes or of other user-defined classes. They may be defined by identifying new attributes or by indicating that optional attributes for the parent class are mandatory for the subclass. The conventions for defining classes defined in 3.8.2 may be used for this purpose. The method for registering or otherwise making these new class definitions available for public use is beyond the scope of this standard.

4.3.4.3 APOs as instances of APO classes

APO Classes are defined in this standard using templates. These templates are used not only to define APO Classes, but also to specify the instances of a class.

Each APO defined for an AP is an instance of an APO Class. Each APO provides the network view of a real object contained in an AP. An APO is defined by

- (1) selecting the attributes from its APO Class template that are to be accessible from the real object;
- (2) assigning values to one or more attributes indicated as key in the template. Key attributes are used to identify the APO;
- (3) assigning values to zero, one, or more non-key attributes for the APO. Non-key attributes are used to characterize the APO;
- (4) selecting the services from the template that may be used by remote APs to access the real object.

Subclause 3.8.2 of this standard specifies the conventions for class templates. These conventions provide for the definition of mandatory, optional, and conditional attributes and services.

Mandatory attributes and services are required to be present in all APOs of the class. Optional attributes and services may be selected, on an APO by APO basis, for inclusion in an APO. Conditional attributes and services are defined with an accompanying constraint statement. Constraint statements specify the conditions that indicate whether or not the attribute is to be present in an APO.

4.3.4.4 APO types

APO types provide the mechanism for defining standard APOs.

As described above in the previous subclauses, APOs are defined by instantiating an APO class. Each APO definition is composed of the attributes and services selected for the APO from those defined by its APO class. In addition, an APO definition contain values for one or more of the attributes selected for it. When two APOs share the same definition, except for

the key attribute settings, that definition is referred to as an APO type. Thus, an APO type is a generic specification of an APO that may be used to define one or more APOs.

4.3.5 Application entities

4.3.5.1 Definition of FAL AE

An application entity provides the communication capabilities for a single AP. An FAL AE provides a set of services and the supporting protocols to enable communications between APs in a fieldbus environment. The services provided by FAL AEs are grouped into Application Service Elements (ASE), such that the FAL services provided to an AP are defined by the ASEs its FAL AE contains. Figure 6 illustrates this concept.

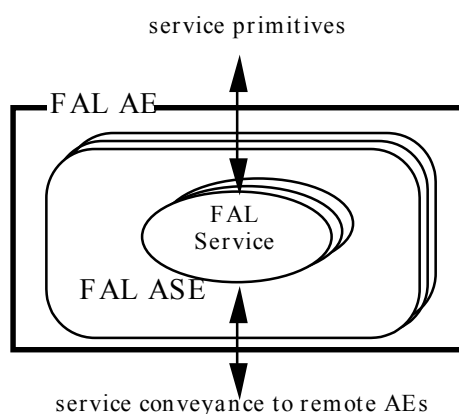


Figure 6 – Application entity structure

4.3.5.2 AE type

Application entities that provide the same set of ASEs are of the same AE-type. Two AEs that share a common set of ASEs are capable of communicating with each other.

4.3.6 Application service elements

4.3.6.1 General

An application service element (ASE), as defined in ISO/IEC 9545, is a set of application functions that provide a capability for the interworking of application-entity-involutions for a specific purpose. ASEs provide a set of services for conveying requests and responses to and from application processes and their objects. AEs, as defined above, are represented by a collection of ASE involutions within the AE.

4.3.6.2 FAL services

FAL Services convey functional requests/responses between APs. Each FAL service is defined to convey requests and responses for access to a real object modeled as an FAL accessible object.

The FAL defines both confirmed and unconfirmed services. Confirmed service requests are sent to the AP containing the real object. An invocation of a confirmed service request may be identified by a user supplied Invoke ID. This Invoke ID is returned in the response by the AP containing the real object. When present, it may be used by the user to associate the response with the appropriate request.

NOTE This user-specified Invoke ID is distinct from any Instance ID that the requesting AP and its FAL AE may itself use to associate request with confirm, indication with response, or command APDU with reply APDU. See 1.2.

Unconfirmed services may be sent from the AP containing the real object to send information about the object. They also may be sent to the AP containing the real object to access the real object. Both types of unconfirmed services may be defined for the FAL.

4.3.6.3 Definition of FAL ASEs

4.3.6.3.1 General

A modular approach has been taken in the definition of FAL ASEs. The ASEs defined for the FAL are also object-oriented. In general, ASEs provide a set of services designed for one specific object class or for a related set of classes. Common object management ASEs, when present, provide a common set of management services applicable to all classes of objects.

To support remote access to the AP, the Application Relationship ASE is defined. It provides services to the AP for defining and establishing communication relationships with other APs, and it provides services to the other ASEs for conveying their service requests and responses.

Each FAL ASE defines a set of services, APDUs, and procedures that operate on the classes that it represents. Only a subset of the ASE services may be provided to meet the needs of an application. Profiles may be used to define such subsets. Definition of profiles is beyond the scope of this standard.

APDUs are sent and received between FAL ASEs that support the same services. Each FAL AE contains, at a minimum, the AR ASE and at least one other ASE. Figure 7 illustrates an example set of the FAL ASEs and their architectural relationships. All APO ASEs follow this example.

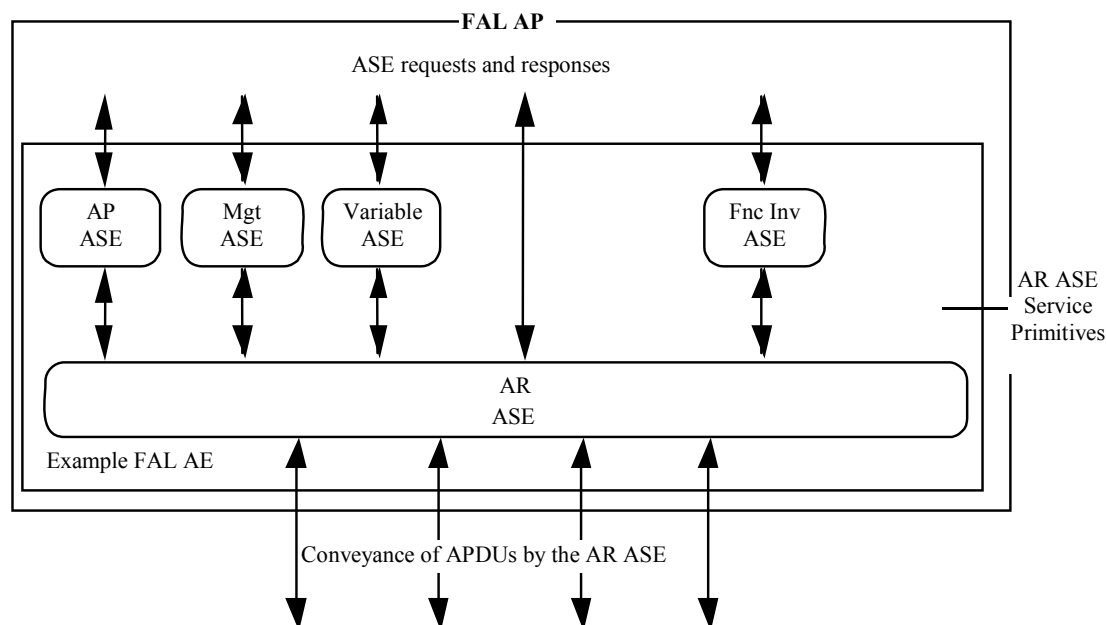


Figure 7 – Example FAL ASEs

4.3.6.3.2 Object management ASE

A special object management ASE may be specified for the FAL to provide services for the management of objects. Its services are used to access object attributes, and create and delete object instances. These services are used to manage network visible AP objects accessed through the FAL. The specific operational services that apply to each object type are

specified in the definition of the ASE for the object type. Figure 8 illustrates the integration of management and operational services for an object within an AP.

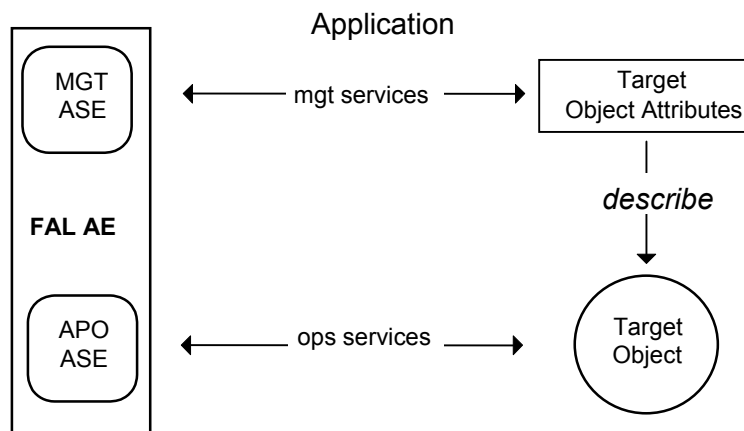


Figure 8 – FAL management of objects

4.3.6.3.3 AP ASE

An AP ASE may be specified for the identification and control of FAL APs. The attributes defined by the AP ASE specify characteristics of the AP about its manufacturer, and list its contents and capabilities.

4.3.6.3.4 APO ASEs

The FAL specifies a set of ASEs with services defined for accessing the APOs of an AP. The APO ASEs defined for the FAL are defined by each Communication Model.

4.3.6.3.5 AR ASE

An AR ASE is specified to establish and maintain application relationships (ARs) that are used to convey FAL APDUs between/among APs. ARs represent application layer communication channels between APs. AR ASEs are responsible for providing services at the endpoints of ARs. AR ASE services may be defined for establishing, terminating, and aborting ARs, for conveying APDUs for the AE, and for indicating the local status of the AR to the user. In addition, local services may be defined for accessing certain aspects of AR endpoints.

4.3.6.4 FAL service conveyance

FAL APO ASEs provide services to convey requests and responses between service users and real objects.

To accomplish the task of conveying service requests and responses, three types of activities for the sending user and three corresponding types for the receiving user are defined. At the sending user, they accept service requests and responses to be conveyed. Second, they select the type of FAL APDU that will be used to convey the request or response and encode the service parameters into its body portion. Then they submit the encoded APDU body to the AR ASE for conveyance.

At the receiving user, they receive encoded APDU bodies from the AR ASE. They decode the APDU bodies and extract the service parameters conveyed by them. To conclude the conveyance, they deliver the service request or response to the user. Figure 9 illustrates these concepts.

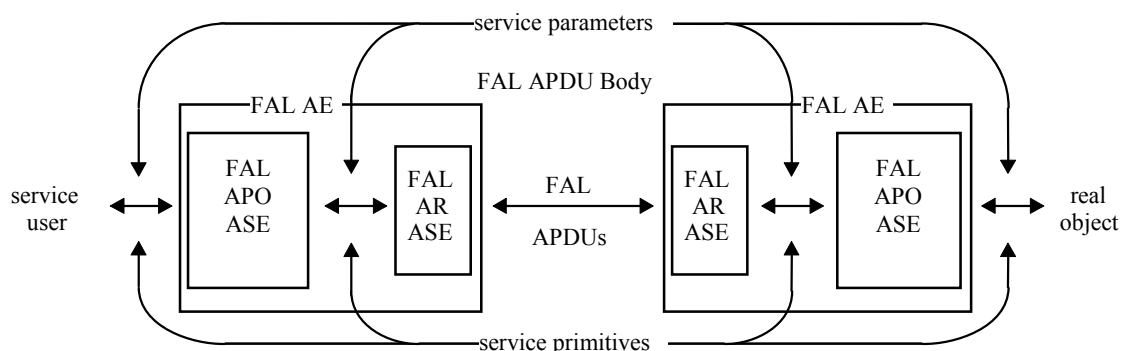


Figure 9 – ASE service conveyance

4.3.6.5 FAL presentation context

The presentation context in the OSI environment is used to distinguish the APDUs of one ASE from another, and to identify the transfer syntax rules used to encode each APDU. However, the fieldbus communications architecture does not include the presentation layer. Therefore, an alternate mechanism is provided for the FAL by each of the specific types of Communication Models.

4.3.7 Application relationships

4.3.7.1 Definition of AR

ARs represent communication channels between APs. They define how information is communicated between APs. Each AR is characterized by how it conveys ASE service requests and responses from one AP to another. These characteristics are described below.

4.3.7.2 AR-endpoints

ARs are defined as a set of cooperating APs. The AR ASE in each AP manages an endpoint of the AR, and maintains its local context. The local context of an AR endpoint is used by the AR ASE to control the conveyance of APDUs on the AR.

4.3.7.3 AR-endpoint classes

ARs are composed of a set of endpoints of compatible classes. AR endpoint classes are used to represent AR endpoints that convey APDUs in the same way. Through the standardization of endpoint classes, ARs for different models of interaction can be defined.

4.3.7.4 AR cardinality

ARs characterize communications between APs. One of the characteristics of an AR is the number of AR endpoints in the AR. ARs that convey services between two APs have a cardinality of 1-to-1. Accessing objects through ARs

ARs provide access to APs and the objects within them through the services of one or more ASEs. Therefore, one characteristic is the set of ASE services that may be conveyed to and from these objects by the AR. The list of services that can be conveyed by the AR are selected from those defined for the AE.

4.3.7.5 AR conveyance paths

ARs are modeled as one or two conveyance paths between AR endpoints. Each conveyance path conveys APDUs in one direction between one or more AR endpoints. Each receiving AR

endpoint for a conveyance path receives all APDUs transmitting on the AR by the sending AR endpoint.

4.3.7.6 AREP roles

Because APs interact with each other through endpoints, a basic determinant of their compatibility is the role that they play in the AR. The role defines how an AREP interacts with other AREPs in the AR.

For example, an AREP may operate as a client, a server, a publisher, or a subscriber. When an AREP interacts with another AREP on a single AR as both a client and a server, it is defined to have the role of “peer”.

Certain roles may be capable of initiating service requests, while others may be capable only of responding to service requests. This part of the definition of a role identifies the requirement for an AR to be capable of conveying requests in either direction, or only in one direction.

4.3.7.7 AREP buffers and queues

AREPs may be modeled as a queue or as a buffer. APDUs transferred over a queued AREP are delivered in the order received for conveyance. The transfer of APDUs over a buffered AREP is different. In this case, an APDU to be conveyed by the AR ASE is placed in a buffer for transfer. When the data-link layer gains access to the network, it transmits the contents of the buffer.

When the AR ASE receives another conveyance request, it replaces the previous contents of the buffer whether or not they were transmitted. Once an APDU is written into a buffer for transfer, it is preserved in the buffer until the next APDU to be transmitted replaces it. While in the buffer, an APDU may be read more than once without deleting it from the buffer or changing its contents.

At the receiving end, the operation is similar. The receiving endpoint places a received APDU into a buffer for access by the AR ASE. When a subsequent APDU is received, it overwrites the previous APDU in the buffer whether or not it was read. Reading the APDU from the buffer is not destructive — it does not destroy or change the contents of the buffer, allowing the contents to be read from the buffer one or more times.

4.3.7.8 User-triggered and scheduled conveyance

Another characteristic of an AREP is when they convey service requests and responses. AREPs that convey them upon submission by the user are called user-triggered. Their conveyance is asynchronous with respect to network operation.

AREPs that convey requests and responses at predefined intervals, regardless of when they are received for transfer are termed scheduled. Scheduled AREPs may be capable of indicating when transferred data was submitted late for transmission, or when it was submitted on time, but transmitted late.

4.3.7.9 Definition and creation of AREPs

AREP definitions specify instances of AREP classes. AREPs may be predefined or they may be defined using a “create” service if their AE supports this capability.

AREPs may be pre-defined and pre-established, or they may be pre-defined and dynamically established. Figure 10 depicts these two cases. AREPs also may require both dynamic definition and establishment or they may be dynamically defined such that they may be used without any establishment (they are defined in an established state).

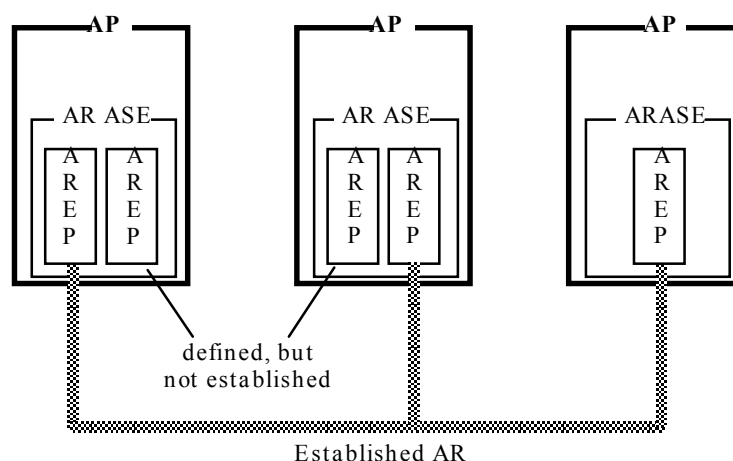


Figure 10 – Defined and established AREPs

4.3.7.10 AR establishment and termination

ARs may be established either before the operational phase of the AP or during its operation. When established during the operation of an AP, the AR is established through the exchange of AR APDUs.

Once an AR has been established, an AR may be terminated gracefully or it may be aborted, depending on the capabilities of the AR.

4.4 FAL naming and addressing

4.4.1 General

This subclause refines the principles defined in ISO 7498-3 that involve the identification (naming) and location (addressing) of APOs referenced through the fieldbus Application Layer.

This subclause defines how names and numeric identifiers are used to identify APOs accessible through the FAL. This subclause also indicates how addresses from underlying layers are used to locate APs in the fieldbus environment.

4.4.2 Identifying objects accessed through the FAL

4.4.2.1 General

APOs accessed through the FAL are identified independent of their location. That is, if the location of the AP that contains the APO changes, the APO may still be referenced using the same set of identifiers.

Identifiers for APs and APOs within the FAL are defined as key attributes in the class definitions for APOs. Within these APO definitions, two types of key attributes are commonly used, names and numeric identifiers.

4.4.2.2 Names

Names are string-oriented identifiers. They are defined to permit APs and APOs to be named within the system where they are used. Therefore, although the scope of the name of an APO is specific to the AP in which it resides, the assignment of the name is administered within the system in which it is configured.

Names may be descriptive, although they do not have to be. Descriptive names make it possible to provide meaningful information, such as its use, about the object they name.

Names may also be coded. Coded names make it possible to identify an object using a short, compressed form of a name. They are typically simpler to transfer and process, but not as easy to understand as descriptive names.

4.4.2.3 Numeric identifiers

Numeric identifiers are identifiers whose values are numbers. They are designed for efficient use within the fieldbus system, and may be assigned for efficient access to APOs by their AP.

4.4.3 Addressing APs accessed through the FAL

Fieldbus addresses represent the network locations of APs. Addresses relevant to the FAL are the addresses of the underlying layers that are used to locate the AREPs of an AP.

4.5 Architecture summary

The summary of the FAL architecture is presented in this Clause. Figure 11 illustrates the major components of the FAL architecture and how they relate to each other.

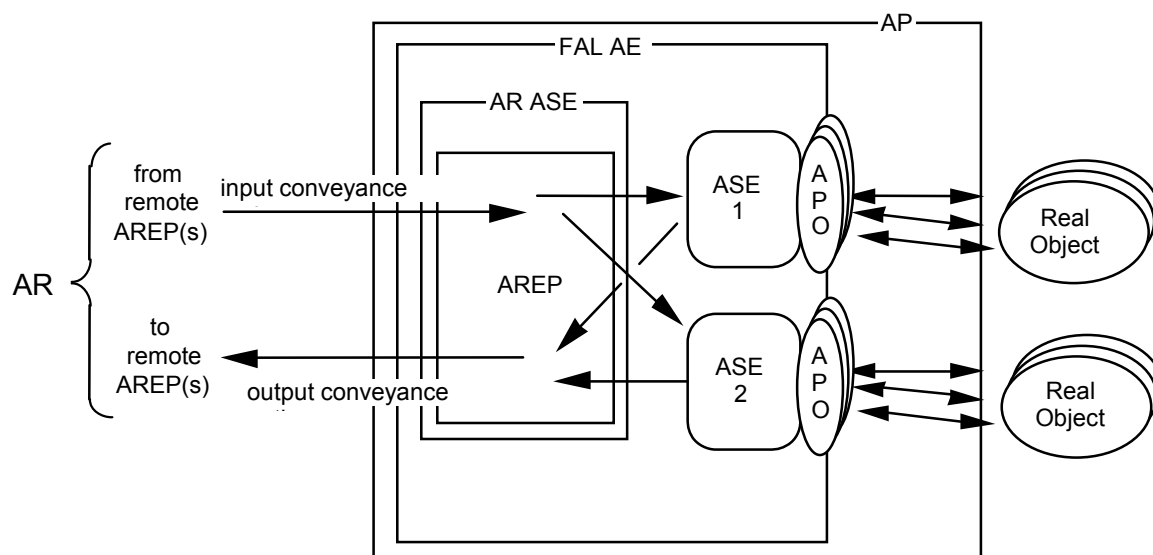


Figure 11 – FAL architectural components

Figure 11 depicts an AP that communicates through the FAL AE. The AP represents its internal real objects as APOs for remote access to them. Two ASEs are shown that provide the remote access services to their related APOs. The AR ASE contains a single AREP that conveys service requests and responses for the ASEs to one or more remote AREPs located in remote APs.

4.6 FAL service procedures

4.6.1 FAL confirmed service procedures

4.7 Notional FAL service procedures [INFORMATIVE]

NOTE This subclause describes one way in which this abstract service definition might be realized in a concrete protocol specification and conforming implementations. See 1.2

4.7.1 Notional FAL confirmed service procedures

The requesting AL-service user invokes a confirmed-service request primitive of its FAL. The appropriate FAL ASE creates a transaction state machine to control the invocation of the service, assigns an InstanceID and timeout time to that state machine, builds the related confirmed-service-request APDU body including that Instance ID, and conveys it on the specified AR.

Upon receipt of the confirmed-service-request APDU body, the receiving ASE decodes it. If a protocol error did not occur, the receiving ASE creates a transaction state machine to manage the expected response, assigns an independent (second) InstanceID to that state machine, then delivers a confirmed-service indication primitive to it's AL-service user, with the (second) InstanceID as an extra implementation parameter.

If the responding AL-service user is able to successfully process the request, the user returns a confirmed-service response (+) primitive, identifying the transaction by the InstanceID presented as part of the stimulating indication primitive.

If the responding user is unable to successfully process the request, the service fails and the user issues a confirmed-service response (–) primitive indicating the reason for failure, again identifying the transaction by the InstanceID presented as part of the stimulating indication primitive.

Whichever response the AL-service users chooses, the receiving ASE has available both the information from the response primitive and that from the associated indication primitive when it forms the APDU to be returned to the initiating ASE.

The responding ASE builds a confirmed-service-response APDU body for a confirmed-service response (+) primitive or a confirmed-service-error APDU body for a confirmed-service response (–) primitive, either of which contains the (first) InstanceID of the original requesting APDU, and conveys it on the specified AR.

Upon receipt of the response or error APDU body, the initiating ASE uses the (first) InstanceID contained in the response or error APDU to associate the APDU with the appropriate state machine and request. Once that association has been made, the initiating ASE has available both the information from the received APDU and that from the associated request primitive. It delivers a confirmed-service confirmation primitive to the requesting FAL ASE which specifies success or failure, reports the reason for failure if a failure occurred, and cancels the associated transaction state machine.

If the timer associated with the state machine expires before the initiating ASE receives the returned response or error APDU, the AR ASE delivers a confirmed-service confirmation(–) primitive to the requesting FAL ASE and cancels the associated transaction state machine.

4.7.2 Notional FAL unconfirmed service procedures

The requesting user submits an unconfirmed service request primitive to its FAL AE. The appropriate FAL ASE builds the related unconfirmed service request APDU Body and conveys it on the specified AR.

Upon receipt of the unconfirmed request APDU Body, the receiving ASE(s) participating in the AR delivers the appropriate unconfirmed service indication primitive to its user. .

4.8 Common FAL attributes

In the specifications of the FAL classes that follow, many classes use the following attributes. Therefore, these attributes are defined here instead of with the other attributes for each of the classes, except for the Data type class.

ATTRIBUTES:

- 1 (o) Key Attribute: Numeric Identifier
- 2 (o) Key Attribute: Name

Numeric identifier

This optional key attribute specifies the numeric id of the object. It is used as a shorthand reference by the FAL protocol to identify the object. There are three possibilities for identification purposes: numeric identifier or name or both. This attribute is required for the Data type model.

Name

This optional key attribute specifies the name of the object. There are three possibilities for identification purposes: numeric identifier or name or both.

4.9 Common FAL service parameters

In the specifications of the FAL services that follow, many services use the following parameters. Therefore, they are defined here instead of with the other parameters for each of the services.

AREP

This parameter specifies sufficient information to locally identify the AREP to be used to convey the service. This parameter may use a key attribute of the AREP to identify the application relationship. When an AREP supports multiple contexts (established using the Initiate service) at the same time, the AREP parameter is extended to identify the context as well as the AREP.

Invoke ID

This user-provided parameter identifies this invocation of the service. It is used to associate service requests with responses. Therefore, no two outstanding service invocations should be identified by the same Invoke ID value.

NOTE 2 The mechanism by which an implementation of these abstract services correlates request and confirm primitives at one AREP, related indication and response primitives at a second AREP, and the APDUs that link remote AEs to implement such services, is a protocol issue as stated in 1.2. Any service Instance ID that might be carried in those APDUs is logically unrelated to this user-provided Invoke ID.

FAL ASE/FAL class

This parameter specifies the FAL ASE (e.g. AP, AR, Variable, Data type, Event, Function Invocation, Load Region) and the FAL Class within the ASE (e.g. AREP, Variable List, Notifier, Action).

Numeric ID

This parameter is the numeric identifier of the object.

Error info

This parameter provides error information for service errors. It is returned in confirmed service response(-) primitives. It is composed of the following elements.

Error class

This parameter indicates the general class of error. Valid values are specified in the definition of Error Code parameter, below.

Error code

This parameter identifies the specific service error.

Additional code

This optional parameter identifies the error encountered when processing the request specific to the object being accessed. When used, the value submitted in the response primitive is delivered unchanged in the confirmation primitive.

4.10 APDU size

APDU size is Communication Model dependent.

5 Data type ASE

5.1 General

5.1.1 Overview

Data types specify the machine independent syntax for application data conveyed by FAL services. The fieldbus application layer supports the definition and transfer of both basic and constructed data types. Encoding rules for the data types specified in this Clause are provided in IEC 61158-6-8.

Basic types are atomic types that cannot be decomposed into more elemental types. Constructed types are types composed of basic types and other constructed types. Their complexity and depth of nesting is not constrained by this standard.

Data types are defined as instances of the data type class, as shown in Figure 12. Only a subset of the data types defined in this Clause are shown in this figure. Defining new types is accomplished by providing a numeric id and supplying values for the attributes defined for the data type class.

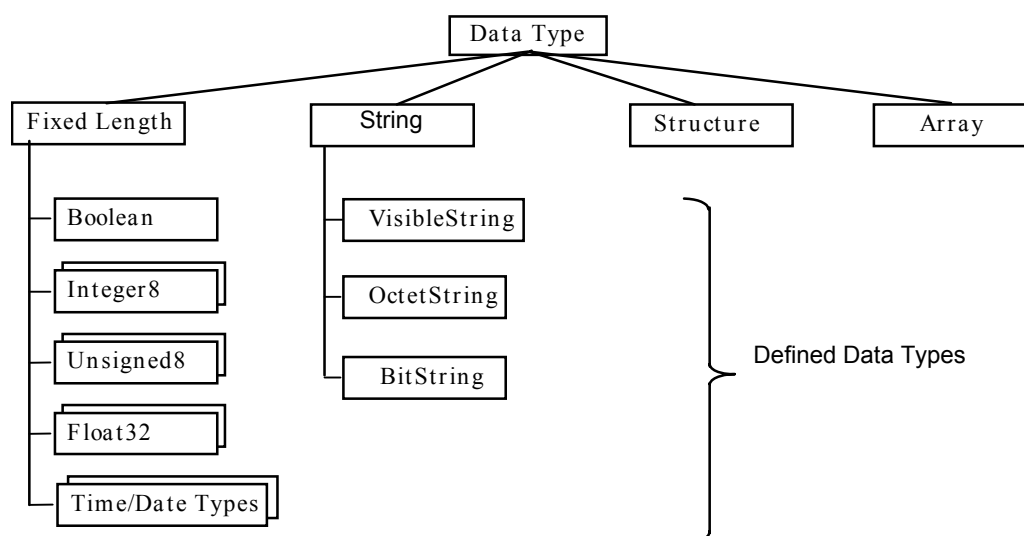


Figure 12 – Data type class hierarchy example

The data type definitions in Figure 12 are represented as a class/format/instance structure beginning with data type class entitled "Data type". The formats for data types are defined by the data type class and are represented in Figure 12.

The basic data classes are used to define fixed length and bitstring data types. Standard types taken from ISO/IEC 8824 are referred to as *simple* data types. Other standard basic data types are defined specifically for fieldbus applications and are referred to as *specific types*.

The constructed types specified in this standard are strings, arrays and structures. There are no standard types defined for arrays and structures.

5.1.2 Basic type overview

Most basic types are defined from a set of ISO/IEC 8824 types (simple types). Some ISO/IEC 8824 types have been extended for fieldbus specific use (specific types).

Simple types are ISO/IEC 8824 universal types. They are defined in this standard to provide them with fieldbus class identifiers.

Specific types are basic types defined specifically for use in the fieldbus environment. They are defined as simple class subtypes.

Basic types have a constant length. Two variations are defined, one for defining data types whose length is an integral number of octets, and one for defining data types whose length is bits.

NOTE Boolean, Integer, OctetString, VisibleString, and UniversalTime are defined in this standard for the purpose of assigning fieldbus class identifiers to them. This standard does not change their definitions as specified in ISO/IEC 8824.

5.1.3 Fixed length type overview

The length of Fixed length types is an integral number of octets.

5.1.4 Constructed type overview

5.1.4.1 Strings

A string is composed of an ordered set, variable in number, of homogeneously typed fixed-length elements.

5.1.4.2 Arrays

An array is composed of an ordered set of homogeneously typed elements. This standard places no restriction on the data type of array elements, but it does require that each element be of the same type. Once defined, the number of elements in an array may not be changed.

5.1.4.3 Structures

A structure is made of an ordered set of heterogeneously typed elements called fields. Like arrays, this standard does not restrict the data type of fields. However, the fields within a structure do not have to be of the same type.

5.1.4.4 Nesting level

Only nesting level 1 is used.

5.1.5 Specification of user defined data types

Users may find it necessary to define custom data types for their own applications. User defined types are supported by this standard as instances of data type classes.

User defined types are specified in the same manner that all FAL objects are specified. They are defined by providing values for the attributes specified for their class.

5.1.6 Transfer of user data

User data is transferred between applications by the FAL protocol. All encoding and decoding are performed by the FAL user.

The rules for encoding user data in FAL protocol data units is data type dependent. These rules are defined in IEC 61158-6-8. User-defined data types for which there are no encoding rules are transferred as a variable-length sequence of octets. The format of the data within the octet string is defined by the user.

5.2 Formal definition of data type objects

5.2.1 Data type class

5.2.1.1 Template

The data type class specifies the root of the data type class tree. Its parent class "top" indicates the top of the FAL class tree.

FAL ASE:	DATA TYPE ASE
CLASS:	DATA TYPE
CLASS ID:	5 (FIXED LENGTH & STRING), 6 (STRUCTURE)
PARENT CLASS:	TOP
ATTRIBUTES:	
1 (m) Key Attribute:	Data type Numeric Identifier
2 (o) Key Attribute:	Data type Name

5.2.1.2 Attributes

Data type Numeric Identifier

This attribute identifies the numeric identifier of the related data type.

Data type Name

This optional attribute identifies the name of the related data type.

5.3 FAL defined data types

5.3.1 Fixed length types

5.3.1.1 Boolean types

5.3.1.1.1 Boolean

CLASS:	Data type
ATTRIBUTES:	
1 Data type Numeric Identifier	= 1
2 Data type Name	= Boolean
3 Format	= FIXED LENGTH
4.1 Octet Length	= 1

This data type expresses a Boolean data type with the values TRUE and FALSE.

5.3.1.2 Date types

5.3.1.2.1 BinaryDate

CLASS:	Data type
ATTRIBUTES:	
1 Data type Numeric Identifier	= 11
2 Data type Name	= BinaryDate

3	Format	=	FIXED LENGTH
4.1	Octet Length	=	7

This data type is composed of six elements of unsigned values and expresses calendar date and time. The first element is an Unsigned16 data type and gives the fraction of a minute in milliseconds. The second element is an Unsigned8 data type and gives the fraction of an hour in minutes. The third element is an Unsigned8 data type and gives the fraction of a day in hours. The fourth element is an Unsigned8 data type. Its upper three (3) bits give the day of the week and its lower five (5) bits give the day of the month. The fifth element is an Unsigned8 data type and gives the month. The last element is Unsigned8 data type and gives the year.

5.3.1.2.2 BinaryDate2000

CLASS:	Data type
ATTRIBUTES:	
1	Data type Numeric Identifier = 51
2	Data type Name = BinaryDate2000
3	Format = FIXED LENGTH
4.1	Octet Length = 8

This data type is composed of six elements of unsigned values and expresses calendar date and time. The first element is an Unsigned16 data type and gives the fraction of a minute in milliseconds. The second element is an Unsigned8 data type and gives the fraction of an hour in minutes. The third element is an Unsigned8 data type and gives the fraction of a day in hours. The fourth element is an Unsigned8 data type. Its upper three (3) bits give the day of the week and its lower five (5) bits give the day of the month. The fifth element is an Unsigned8 data type and gives the month. The last element is Unsigned16 data type and gives the year.

5.3.1.2.3 TimeOfDay

CLASS:	Data type
ATTRIBUTES:	
1	Data type Numeric Identifier = 12
2	Data type Name = TimeOfDay
4	Format = FIXED LENGTH
4.1	Octet Length = 6

This data type is composed of two elements of unsigned values and expresses the time of day and the date. The first element is an Unsigned32 data type and gives the time after the midnight in milliseconds. The second element is an Unsigned16 data type and gives the date counting the days from January 1, 1984.

5.3.1.2.4 TimeDifference

CLASS:	Data type
ATTRIBUTES:	
1	Data type Numeric Identifier = 13
2	Data type Name = TimeDifference
3	Format = FIXED LENGTH
4.1	Octet Length = 4 or 6

This data type is composed of two elements of unsigned values that express the difference in time. The first element is an Unsigned32 data type that provides the fractional portion of one day in milliseconds. The optional second element is an Unsigned16 data type that provides the difference in days.

5.3.1.3 Numeric types

5.3.1.3.1 Floating Point types

5.3.1.3.1.1 Float32

CLASS:	Data type
ATTRIBUTES:	
1 Data type Numeric Identifier	= 8
2 Data type Name	= Float32
4 Format	= FIXED LENGTH
4.1 Octet Length	= 4

This type has a length of four octets. The format for float32 is that defined by ISO 60559 as single precision.

5.3.1.3.2 Integer types

5.3.1.3.2.1 Integer8

CLASS:	Data type
ATTRIBUTES:	
1 Data type Numeric Identifier	= 2
2 Data type Name	= Integer8
3 Format	= FIXED LENGTH
4.1 Octet Length	= 1

This integer type is a two's complement binary number with a length of one octet.

5.3.1.3.2.2 Integer16

CLASS:	Data type
ATTRIBUTES:	
1 Data type Numeric Identifier	= 3
2 Data type Name	= Integer16
3 Format	= FIXED LENGTH
4.1 Octet Length	= 2

This integer type is a two's complement binary number with a length of two octets.

5.3.1.3.2.3 Integer32

CLASS:	Data type
ATTRIBUTES:	
1 Data type Numeric Identifier	= 4
2 Data type Name	= Integer32
3 Format	= FIXED LENGTH
4.1 Octet Length	= 4

This integer type is a two's complement binary number with a length of four octets.

5.3.1.3.3 Unsigned types

5.3.1.3.3.1 Unsigned8

CLASS:	Data type
ATTRIBUTES:	
1 Data type Numeric Identifier	= 5
2 Data type Name	= Unsigned8
3 Format	= FIXED LENGTH
4.1 Octet Length	= 1

This type is a binary number. The most significant bit of the most significant octet is always used as the most significant bit of the binary number; no sign bit is included. This type has a length of one octet.

5.3.1.3.3.2 Unsigned16

CLASS:		Data type
ATTRIBUTES:		
1	Data type Numeric Identifier	= 6
2	Data type Name	= Unsigned16
3	Format	= FIXED LENGTH
4.1	Octet Length	= 2

This type is a binary number. The most significant bit of the most significant octet is always used as the most significant bit of the binary number; no sign bit is included. This unsigned type has a length of two octets.

5.3.1.3.3.3 Unsigned32

CLASS:		Data type
ATTRIBUTES:		
1	Data type Numeric Identifier	= 7
2	Data type Name	= Unsigned32
3	Format	= FIXED LENGTH
4.1	Octet Length	= 4

This type is a binary number. The most significant bit of the most significant octet is always used as the most significant bit of the binary number; no sign bit is included. This unsigned type has a length of four octets.

5.3.2 String types

5.3.2.1 BitString

CLASS:		Data type
ATTRIBUTES:		
1	Data type Numeric Identifier	= 14
2	Data type Name	= Bitstring
3	Format	= STRING
5.1	Octet Length	= 1 to n

This string type is defined as a series of BitString8 elements.

5.3.2.2 OctetString

CLASS:		Data type
ATTRIBUTES:		
1	Data type Numeric Identifier	= 10
2	Data type Name	= OctetString
3	Format	= STRING
4.1	Octet Length	= 1 to n

An OctetString is an ordered sequence of octets, numbered from 1 to n. For the purposes of discussion, octet 1 of the sequence is referred to as the first octet. The order of transmission is defined in IEC 61158-6-8.

5.3.2.3 VisibleString

CLASS:		Data type
ATTRIBUTES:		
1	Data type Numeric Identifier	= 9
2	Data type Name	= VisibleString
3	Format	= STRING
4.1	Octet Length	= 1 to n

This type is defined as the ISO/IEC 646 string type.

5.4 Data type ASE service specification

There are no operational services defined for the type object.

6 Communication model specification

6.1 ASEs

6.1.1 Object management ASE

6.1.1.1 Overview

The FAL ASE Models each specify one or more related FAL APO classes as a collection of attributes and services. The FAL services defined for a class are used to manipulate, control, or access APOs of the class. The services which are supported by a specific APO are specified by the ListOfManagementServices attribute of the APO.

6.1.1.2 FAL management model specification

The management services specified by the FAL Management Model operate on FAL APOs and their attributes. FAL APO classes are defined within the FAL ASEs that are specified in the remaining clauses of this standard. Each FAL APO Class defined is described by a set of attributes and a set of services.

6.1.1.3 FAL management model services

6.1.1.3.1 Supported services

Management services are defined for managing APs and APOs. For simplicity APs and APOs are referred to as objects in the remainder of this subclause.

Get Attributes service is specified to indicate when its attributes can be read.

6.1.1.3.2 Get attributes service

6.1.1.3.2.1 Service overview

This confirmed service is used to read the current values of a list of attributes for one or more objects of one or more classes defined for the AP. It operates in a "best effort" fashion, such that the service succeeds if the value for at least one requested attribute for one requested object is returned.

6.1.1.3.2.2 Service parameters

The service parameters for this service are shown in Table 1.

Table 1 – Get attributes service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument				
AREP	M	M		
Invoke ID	U			
List of Objects and their Attributes	S	S (=)		
Key Attribute	M	M (=)		
List of Requested Attributes	M	M (=)		
Result(+)			S	S (=)
Invoke ID				U (=)
More			M	M (=)
List of Responses			M	M (=)
List of Attribute Values			S	S (=)
Result(-)			S	S (=)
Invoke ID				U (=)
Error Info			M	M (=)
NOTE The method by which a confirm primitive is correlated with its corresponding preceding request primitive is a local matter. See 1.2.				

Argument

The argument contains the parameters of the service request.

List of objects and their attributes

This selection type parameter provides a list that identifies each object, its class, and the attributes that are being requested. Three values for identifying attributes are possible, all attributes, mandatory attributes only, or a list of individual attributes. It is present if the user is requesting attributes for a list of objects.

Key attribute

This parameter identifies the object for which attributes are being requested using one of the object's key attributes.

List of requested attributes

This parameter identifies one or more of the attributes for which values are being requested.

Result(+)

This selection type parameter indicates that the service request succeeded.

More

This Boolean parameter indicates, when FALSE, that responses for all requested attributes are being returned. If the value is TRUE, then only responses for a proper subset of the attributes are being returned. This situation will occur only when not all values will fit into a single response APDU. When it does occur, it is necessary for the user to submit an additional request for the missing attributes. Partial values for attributes are never returned.

List of responses

This parameter specifies a status indicator or a list of attribute values for each object specified in the request. The responses in the list are returned in the same order that the objects were specified in the request. The object associated with each response is only explicitly identified if the value of one of its key attributes was requested. If all responses could not be returned in a single APDU, the More parameter is set.

Error status

This parameter is returned if the get failed for all of the attributes of an object. It indicates the most probable reason why the operation failed using the error class and error code defined for the Get Attributes service.

List of attribute values

This parameter is returned if the Get succeeded for any one or the requested attributes. This parameter specifies a list of attribute values, each individually identified and complete, for the object requested.

Result(-)

This selection type parameter indicates that the service request failed.

6.1.1.3.2.3 Service procedure

The Confirmed Service Procedure specified in 4.6 applies to this service.

The Get Attributes Service is a "best effort". Best effort means that the service succeeds if at least one value is returned. If the user is able to get one or more of the specified attribute values, and if they can be conveyed in a single APDU, they are returned in a Get Attributes Service response (+) primitive.

If they cannot be conveyed in a single APDU, the user returns those that can and sets the more flag in the Get Attributes Service response (+) primitive. The requester is then responsible for submitting additional requests identifying the remaining attributes to be retrieved.

If the user is unable to get the value for at least one attribute for one object in the list, the service fails and the user issues a Get Attributes Service response (-) primitive indicating the reason.

6.1.2 Application process ASE

6.1.2.1 Overview

This standard models an Application Process (AP). Application Processes represent the information and processing resources of a system that can be accessed through FAL services.

The Application Service Element in the FAL that provides these services is called an Application Process ASE. In the AP ASE, the AP is modeled and accessed as an APO with a standardized and predefined identifier.

3.1.3.3 (m)	Attribute:	Static Object Local Reference
3.1.5 (c)	Constraint	Function Invocation Supported
3.1.5.1 (o)	Attribute:	Function Invocation Numeric Identifier
3.1.5.2 (m)	Attribute:	Number Of Function Invocation Objects
3.1.5.3 (m)	Attribute:	Function Invocation Local Reference

SERVICES:

2	(o)	Ops Service:	Identify
3	(o)	Ops Service:	Get Status
5	(o)	Ops Service:	Initiate
6	(o)	Ops Service:	Terminate
8	(o)	Ops Service:	Reject

6.1.2.2.2 Identify, get status service attributes

List of AP service attributes

The following attributes are accessible using the Identify, Get Status Services. They are not otherwise accessible.

Manufacturer identifier

The Manufacturer Identifier is composed of the Vendor Name, Model Identifier, Vendor Revision, and optionally, the Serial Number of the AP. These attributes may be read using the Identify service.

Vendor name

This attribute specifies the name of the manufacturer of the AP.

Model identifier

This attribute specifies the model of the AP.

Vendor revision

This attribute specifies the revision level of the AP as defined by the manufacturer.

AP descriptor reference

This attribute is a reference to a generic description of the AP. The descriptor is an identifier for the characteristics of the AP independent of its use. Two APs, therefore, may share the same generic description. The value, and the permissible set of values for this attribute is user defined. This attribute is used in the Initiate service.

Network access state

The Network Access State attribute defines the ability of the AP to communicate. Two components are specified in this standard, Service Access Status and Operational Status. These attributes may be read using the Get Status service. The AP may choose to report them without being requested using the Status Notification service.

Service access status

This attribute specifies information about the state of the communication capabilities of the AP.

READY FOR COMMUNICATION All services may be used normally.

LIMITED NUMBER OF SERVICES Limited number of services may be supported in some cases (e.g. for small devices).

LOADING-NON-INTERACTING Object Definitions are in the process of being loaded locally. Therefore, the Begin Set Attributes service may not be used until the local load is complete and the state has been changed.

LOADING-INTERACTING Object Definitions are in the process of being loaded over the network (using the Set Attributes service). On the AR used for the loading, only the following services are used:

Abort	Get Attributes
Status	Set Attributes
Identify	End Set Attributes

OperationalStatus

This attribute gives the operational state of the real AP, as follows.

OPERATIONAL

PARTIALLY OPERATIONAL

NOT OPERATIONAL

NEEDS MAINTENANCE

6.1.2.2.3 System management attributes

List of system management attributes

The following attributes are accessible through System Management. They are used by all ASEs of the AP to enforce confirmed service flow control (see the state machines in IEC 61158-6-8). They are not otherwise accessible.

List of AR Endpoint Info

This attribute specifies the numeric identifiers and other configuration information (the AP Context attributes) of AREPs associated with the AP.

Numeric ID

This attribute specifies the numeric id for the AREP.

Configured max FAL PDU size sending

For non-segmenting ARs, this attribute specifies the configured maximum number of octets that can be sent from this AREP. Its value is equal to the maximum DLSDU size minus one that may be sent from this AREP. For segmenting ARs, it specifies the maximum number of 256-octet segments that can be sent on this AREP.

Configured max FAL PDU size receiving

For non-segmenting ARs, this attribute specifies the configured maximum number of octets that can be received on this AREP. Its value is equal to the maximum DLSDU size minus one that may be received on this AREP. For segmenting ARs, it specifies the maximum number of 256-octet segments that can be received on this AREP.

Configured max outstanding services requesting (ConfigMaxOsReq)

This attribute specifies the maximum number of outstanding confirmed services allowed in the client role of this AREP. Its value is configured before establishment of the AR. This attribute is used on client-server and peer-to-peer ARs (i.e. QUB) and corresponds to the number of sending transaction state machines.

Configured max outstanding services responding (ConfigMaxOsRsp)

This attribute specifies the maximum number of outstanding confirmed services allowed for this AREP as a server. Its value is configured before establishment of the AR. This attribute is used on client-server and peer-to-peer ARs (i.e., QUB) and corresponds to the number of receiving transaction state machines.

List of supported services

This attribute specifies the services that the FAL supports as a requester and as a responder. Support as a requester indicates that the FAL provides the ability for the AP to initiate confirmed and unconfirmed request primitives and to receive corresponding confirmed service confirmation primitives from the FAL. Support as a responder indicates that the FAL provides the ability for the AP to deliver confirmed and unconfirmed service indication primitives to the AP and receive confirmed service response primitives from the AP.

List of in-use AR endpoint info

This attribute specifies dynamic information for the AREPs associated with the AP which are participating in an established AR-I or which are involved in the AR establishment process.

Context state

This attribute specifies the state of the AREP as seen by the AP. The values are:

CLOSED

OPEN

OPENING-REQUESTING

OPENING-RESPONDING

Outstanding services requesting counter (OsReqCtr)

This attribute specifies the number of currently outstanding confirmed services on this AR as a requester.

Outstanding services responding counter (OsRspCtr)

This attribute specifies the number of currently outstanding confirmed services on this AR as a responder.

6.1.2.2.4 Object management attributes

List of managed AP attributes

This attribute specifies the AP attributes that are accessible using the services of the Management ASE.

List of supported APO classes and objects

This attribute specifies the Object Definitions maintained by the AP.

List header

This attribute describes the characteristics of the Object Definitions maintained by the AP.

Numeric ID

This attribute is the Numeric ID of the List Header. Its value is always 0.

ROM / RAM flag

This attribute, when TRUE, indicates that modifications to the Object Definitions are allowed.

Max name length

This attribute specifies the maximum length in octets for names of objects defined for this AP.

Access protection supported

This attribute, when TRUE, indicates that Access Protection is supported.

Version of list

This attribute indicates the current version of this list. Each update to the list of object definitions, either locally initiated or through the use of the SetAttributes service causes the version number to be incremented.

List local reference

This attribute is a reference to the list that has local significance, such as an address. The value FFFFFFFF hex indicates that no local reference is available.

XYZ object class supported constraint

This constraint selects object classes specified by XYZ. The attributes following the constraint apply to Object Definitions of the selected object class.

XYZ numeric ID

This attribute is the Numeric ID of the first object in a series of objects of the class(es) selected by the constraint. All objects of the selected class(es) are contained in the series. The first Numeric ID for Data types is always 1.

Number of XYZ entries

This attribute specifies the number of objects in the series.

XYZ local reference

This attribute is a reference to the beginning of object definitions that describe the objects in the series. The reference has local significance, such as an address. The value FFFFFFFF hex indicates that no local reference is available.

6.1.2.2.5 Services**Identify**

This optional service is used to request manufacturer information from the AP.

Get Status

This optional service is used to request the network visible state from the AP.

Initiate

This optional service is used to open the AP context for an Application Relationship.

Terminate

This optional service is used to abruptly close the AP context for an Application Relationship.

Reject

This service is initiated internally by the AP ASE to indicate that a confirmed service request APDU has been received with a protocol error.

6.1.2.3 Application process ASE service specification

6.1.2.3.1 Supported services

This subclause contains the definition of services that are unique to this ASE. The services defined for this ASE are:

Identify

Get Status

Initiate

Terminate

Reject

6.1.2.3.2 Identify service

6.1.2.3.2.1 Service overview

Information to identify an AP is read with this service.

NOTE The attribute ProfileNumber of the AP is transmitted with the Initiate service.

6.1.2.3.2.2 Service primitives

The service parameters for this service are shown in Table 2.

Table 2 – Identify

Parameter name	Req	Ind	Rsp	Cnf
Argument				
AREP	M	M		
Invoke ID	U			
Result(+)			S	S (=)
Invoke ID				U (=)
Vendor Name			M	M (=)
Model Identifier			M	M (=)
Vendor Revision			M	M (=)
Result(-)			S	S (=)
Invoke ID				U (=)
Error Info			M	M (=)
NOTE The method by which a confirm primitive is correlated with its corresponding preceding request primitive is a local matter. See 1.2.				

Argument

The argument contains the parameters of the service request.

Result(+)

This selection type parameter indicates that the service request succeeded.

Vendor name

This parameter specifies the value of the Vendor Name attribute of the AP.

Model identifier

This parameter specifies the value of the Model Identifier attribute of the AP.

Vendor revision

This parameter specifies the value of the Vendor Revision attribute of the AP.

Result(-)

This selection type parameter indicates that the service request failed.

6.1.2.3.2.3 Service procedure

The Confirmed Service Procedure specified in 4.6 applies to this service.

6.1.2.3.3 Get status service**6.1.2.3.3.1 Service overview**

The device/user state is read with the Get Status service.

6.1.2.3.3.2 Service primitives

The service parameters for this service are shown in Table 3.

Table 3 – Get status

Parameter name	Req	Ind	Rsp	Cnf
Argument				
AREP	M	M		
Invoke ID	U			
Result(+)			S	S (=)
Invoke ID				U (=)
Service Access Status			M	M (=)
Operational Status			M	M (=)
Local Detail			U	U (=)
Result(-)			S	S (=)
Invoke ID				U (=)
Error Info			M	M (=)
NOTE The method by which a confirm primitive is correlated with its corresponding preceding request primitive is a local matter. See 1.2.				

Argument

The argument contains the parameters of the service request.

Result(+)

This selection type parameter indicates that the service request succeeded.

Service access status

This parameter specifies the value of the Service Access Status attribute of the AP.

Operational status

This parameter specifies the value of the Operational Status attribute of the AP.

Local detail

This parameter specifies the user defined state of the application.

Result(-)

This selection type parameter indicates that the service request failed.

6.1.2.3.3.3 Service procedure

The Confirmed Service Procedure specified in 4.6 applies to this service.

6.1.2.3.4 Initiate service**6.1.2.3.4.1 Service overview**

This service is used to establish a context between communicating APs for the exchange information on a single AR. The Initiate service negotiates the supported FAL services, the maximum outstanding services permitted, the maximum PDU length and the current version of the Object Definitions. The supported FAL services, the maximum outstanding services permitted, the maximum PDU length are AP system management attributes configured for the AREP.

6.1.2.3.4.2 Service primitives

The service parameters for this service are shown in Table 4.

Table 4 – Initiate

Parameter name	Req	Ind	Rsp	Cnf
Argument				
AREP	M	M		
Version Object Definitions Calling	M	M (=)		
AP Descriptor Calling	M	M (=)		
Access Protection Supported Calling	M	M (=)		
Password Calling	M	M (=)		
Access Groups Calling	M	M (=)		
Result(+)			S	S (=)
Version OD Called			M	M (=)
AP Descriptor Called			M	M (=)
Access Protection Supported Called			M	M (=)
Password Called			M	M (=)
Access Groups Called			M	M (=)
Result(-)			S	S (=)
Error Code			M	M (=)
Max PDU Length Sending Called				C
Max PDU Length Receiving Called				C
FAL Features Supported Called				C
NOTE The method by which a confirm primitive is correlated with its corresponding preceding request primitive is a local matter. See 1.2.				

Argument

This parameter carries the parameters of the service invocation.

Version object definitions calling

This parameter specifies the version of the client's Object Definitions. Its value is null if the client does not contain Object Definitions.

AP descriptor calling

This parameter specifies the value of the calling AP's Descriptor Reference attribute if it has one. If it does not, its value is null.

Access protection supported calling

This parameter specifies the value of the calling AP's Access Protection Supported attribute if it has one. If it does not, its value is null.

Password calling

This parameter specifies the password to be used for the access to all objects of the server on this AR. If access with password is not to be used on this AR, its value is null.

Access groups calling

This parameter specifies the client's membership in specific access groups. The membership applies for the access to all objects of the server on this AR.

Result(+)

This selection type parameter indicates that the service request succeeded.

Version object definition called

This parameter specifies the version of the server's Object Definitions.

AP descriptor called

This parameter specifies the value of the called AP's Descriptor Reference attribute.

Access protection supported called

This parameter specifies the value of the called AP's Access Protection Supported attribute if it has one. If it does not, its value is null.

Password called

This parameter specifies the password to be used for the access to all objects of the client on this AR. If access with password is not to be used on this AR, its value is null.

Access groups called

This parameter specifies the server's membership in specific access groups. The membership applies for the access to all objects of the client on this AR.

Result(-)

This selection type parameter indicates that the service request failed.

Error code

This parameter provides the reason for the failure.

Reason	Meaning
Max FAL PDU Size Insufficient	The maximum PDU length is not sufficient for the communication.
Service Not Supported	The requested service or option is not supported by the server.
User Initiate Denied	The FAL user refuses to establish the connection.
Version Object Definition incompatible	The called and calling versions of the Object Definitions are not compatible.
Password Error	There is already an AR established with the same password, or the password is not valid.
AP Descriptor incompatible	The descriptor is not supported by the server.
Other	Reason other than any of those identified above.

Max PDU length sending called

This parameter specifies the maximum length of the FAL PDU to be sent that may be handled on this AR.

It is transmitted by the called FAL and is part of the Initiate.cnf primitive.

Max PDU length receiving called

This parameter specifies the maximum length of the FAL PDU to be received that may be handled on this AR.

If supported, it is transmitted by the called FAL and is part of the Initiate.cnf primitive.

FAL services supported called

This parameter identifies the optional AL services and the options supported by the server (see AR List).

If supported, it is transmitted by the called FAL and is part of the Initiate.cnf primitive.

6.1.2.3.4.3 Service procedure

This service operates through a queue.

The requesting user submits the service request primitive to its FAL AE and starts an associated timer to monitor the request. The AP ASE builds the Initiate Request APDU Body and conveys it on the specified AR using the AR Establish Service request primitive. It also creates a transaction state machine.

Upon receipt of the Initiate Request APDU Body in an AR Establish service indication primitive, the responding AP ASE decodes it. If a protocol error was encountered while decoding the received APDU Body, the ASE uses the AR Abort Service to abort the AR. If a protocol error did not occur, the ASE delivers an Initiate service indication primitive to its user.

If the responding user is able to successfully process the request, the user returns an Initiate service response (+) primitive.

If the responding user is unable to successfully process the request, the service fails and the user issues an Initiate service response (-) primitive indicating the reason.

The responding AP ASE builds an Initiate service response APDU Body for a response (+) primitive or an Initiate service error APDU Body for a response (-) primitive and conveys it on the specified AR using an AR Establish service response primitive.

Upon receipt of the returned APDU Body in an AR Establish service confirmation (+ or -) primitive the initiating ASE delivers an Initiate service confirmation primitive to its user which specifies success or failure, and the reason for failure if a failure occurred. It also cancels the corresponding transaction state machine.

However, if the AP's transaction timer expires before the corresponding response or error APDU is received, the AP may take corrective actions, which may include instructing its local ASE to cancel the transaction state machine. Such instruction to the ASE would occur through a locally defined interface.

6.1.2.3.5 Terminate service

6.1.2.3.5.1 Service overview

This service is used to release an existing AR between APs, or to terminate a subscriber/receiver AREP's involvement in an AR.

6.1.2.3.5.2 Service primitives

The service parameters for this service are shown in Table 5.

Table 5 – Terminate

Parameter name	Req	Ind
Argument		
AREP	M	M
Locally Generated		M
Terminate Identifier	M	M (=)
Reason Code	M	M (=)
Terminate Detail	U	U (=)

Argument

This parameter carries the parameters of the service invocation.

Locally generated

This parameter indicates whether the termination was generated locally or by the communication partner.

The value false is not allowed if the Terminate Identifier parameter has the value FAL and the Reason Code parameter has the value AR Error.

Terminate identifier

This parameter indicates where the reason for the termination has been detected. Possible values are:

FAL USER
FAL APO ASE
FAL AR ASE
DLL

Reason code

This parameter specifies the reason for the termination.

If the Terminate Identifier parameter has the value USER, the following values apply.

	Meaning
Disconnection	The connection is released by the FAL user.
Version Object Definition incompatible	The called and calling versions of the Object Definitions are not compatible.
Password Error	There is already an AR established with the same password, or the password is not valid.
Descriptor incompatible	The server's descriptor is not supported by the client.
Limited Services Permitted	The AP is in the Logical Status LIMITED-SERVICES-PERMITTED.

If the Abort Identifier parameter has the value FAL, the following values apply:

Reason	Meaning
AR Error	Faulty AR
User Error	Improper, unknown or faulty service primitive received from the FAL user
FAL PDU Error	Unknown or faulty FAL PDU received from the AR ASE
Connection State Conflict AR ASE	Improper AR ASE service primitive
AR ASE Error	Unknown or faulty AR ASE service primitive
PDU Size	PDU length exceeds maximum PDU length
Feature Not Supported	SERVICE-REQ_PDU received from the AR ASE and service or option not supported as a server (see FAL Features Supported attribute in the AR)
Invoke ID Error Response	Confirmed service.rsp received from the FAL user and Invoke ID does not exist or CONFIRMED_SERVICE-RSP_PDU received from the AR ASE and Invoke ID does not exist
Max Services Overflow	CONFIRMED_SERVICE-REQ_PDU received from the AR ASE and Outstanding Services Counter Receiving ≥ Max Outstanding Services Receiving
Connection State Conflict Service Error	INITIATE-REQ_PDU received from the AR ASE The service in the response does not match the service in the indication or the service in the confirmation does not match the service in the request.
Invoke ID Error Request	CONFIRMED_SERVICE-REQ_PDU received from the AR ASE and Invoke ID already exists.

If the Terminate Identifier parameter has the value AR ASE or DLL, the Reason Code parameter is supplied by the AR ASE.

Terminate detail

This optional parameter specifies additional information about the abort reason (max. 16 octets). In case of error reports from the application, the meaning is defined in the profile.

6.1.2.3.5.3 Service procedure

The Unconfirmed Service Procedure specified in 4.6 applies to this service.

6.1.2.3.6 Reject service

6.1.2.3.6.1 Service overview

This service is used to inform the remote endpoint that a protocol error has been detected. It is generated by the AP ASE if an APDU has been received with an invalid service type code. When another APO ASE detects a protocol error, it internally requests the AP ASE to generate a Reject PDU.

6.1.2.3.6.2 Service primitives

The service parameters for this service are shown in Table 6.

Table 6 – Reject

Parameter name	Req	Ind
Argument		
AREP	M	M
Reject Code	M	M (=)
Original FAL Service Type	M	M (=)
Original Invoke ID	M	M (=)
Original PDU Body	U	U (=)

Argument

The argument contains the parameters of the service request.

Reject code

This parameter indicates the reason for rejection. The values of this may indicate:

Unsupported Service

AR Not Established

AR Not Defined

Max Outstanding Requests Exceeded

Max PDU Size Exceeded

Duplicate Invoke ID

Original FAL service type

This parameter specifies the service type of the rejected service request.

Original invoke ID

This parameter specifies the invoke id of the rejected service request.

Original PDU body

This optional parameter specifies some or all of the data of the APDU that was rejected. The amount of data included is determined by the user.

6.1.2.3.6.3 Service procedure

The Reject service is a service that operates through a queue or buffer. It may be initiated only by the FAL AP ASE.

If any APO ASE receives a confirmed request APDU that contains a protocol error, the AP ASE builds a Reject Request APDU and returns it on the specified AR.

Upon receipt of the Reject Request APDU, the receiving AP ASE delivers an AR-Reject.indication primitive through the FAL ASE that issued the confirmed send request primitive, and it issues a negative confirmation to the requesting user.

6.1.3 Application relationship ASE

6.1.3.1 Overview

6.1.3.1.1 General

In a distributed system, application processes communicate with each other by exchanging application layer messages across well-defined application layer communications channels. These communication channels are modeled in the FAL as application relationships (ARs).

ARs are responsible for conveying messages between applications according to specific communications characteristics required by time-critical systems. Different combinations of these characteristics lead to the definition of different types of ARs. The characteristics of ARs are defined formally as attributes of AR Endpoint classes.

The messages that are conveyed by ARs are FAL service requests and responses. Each is submitted to the AR ASE for transfer by an FAL Application Service Element (ASE) that represents the class of the APO being accessed. Figure 13 illustrates this concept.

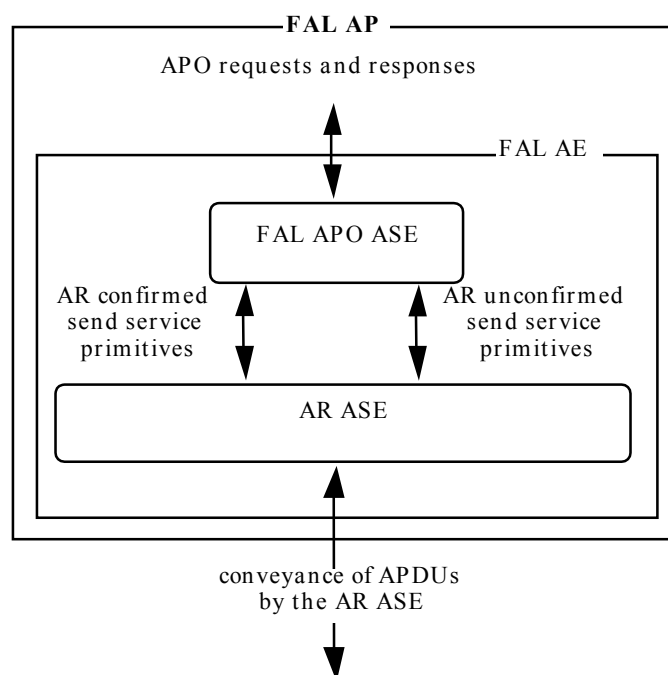


Figure 13 – The AR ASE conveys APDUs between APs

Depending on the type of AR, APDUs may be sent to one or more destination application processes connected by the AR. Other characteristics of the AR determine how APDUs are to be transferred. These characteristics are described below.

6.1.3.1.2 Endpoint context

6.1.3.1.2.1 General

Each AP involved in an AR contains an endpoint of the AR. Each AR endpoint is defined within the AE of the AP. Its definition, when combined with the definitions of the other endpoints defines an AR. To ensure communications compatibility among or between endpoints, each endpoint definition contains a set of compatibility-related characteristics. These characteristics need to be configured appropriately for each endpoint for the AR to operate properly.

Endpoint definitions also contain a set of characteristics that describe the operation of the AR. These characteristics, when combined with those used to specify compatibility, define the context of the endpoint. The endpoint context is used by the AR ASE to manage the operation of the endpoint and the conveyance of APDUs. The characteristics which comprise the endpoint context are described next.

6.1.3.1.2.2 Endpoint role

The role of an AREP determines the permissible behavior of an AP at the AREP. An AREP role may be that of a client, server, peer (client and/or server), push publisher, push subscriber.

Table 7 and Table 8 summarize the characteristics and combinations of each of the AREP roles.

Table 7 – Conveyance of service primitives by AREP role

	Client	Server	Peer	Push Publisher	Push Subscriber
Send Service Req	X		X	X	
Recv Service Req		X	X		X
Send Service Rsp		X	X		
Recv Service Rsp	X		X		

Table 8 – Valid combinations of AREP roles involved in an AR

AREP Roles by Interaction Model	Client	Server	Peer	Push Publisher	Push Subscriber
Client/server					
Client		Yes	Yes		
Server			Yes		
Peer			Yes		
Publisher/subscriber					
Push Publisher					Yes
Push Subscriber					

6.1.3.1.2.3 Dedicated AR endpoints

AR endpoints that are dedicated provide their services directly to the FAL User. Although their behavior is the same as non-dedicated endpoints, the APDUs they convey contain no service specific protocol control information other than the AR control field.

FAL Users configured for dedicated ARs construct and transfer their data without using the services of the other FAL ASEs. The format and contents of the user data conveyed over dedicated ARs are known only through configuration.

6.1.3.1.2.4 Cardinality

The cardinality of an AR specifies, from the point of view of a client or publisher endpoint, how many remote application processes are involved in an AR. Cardinality is never expressed from the viewpoint of a server or a subscriber.

When expressed from the viewpoint of a client or peer endpoint, ARs are always 1-to-1. Clients are never capable of issuing a request and waiting for responses from multiple servers.

6.1.3.1.2.5 Conveyance model

6.1.3.1.2.5.1 General

The conveyance model defines how APDUs are sent between endpoints of an AR. Three characteristics are used to define these transfers:

- conveyance paths
- trigger policy, and
- conveyance policy.

6.1.3.1.2.5.2 Conveyance paths

The purpose of AR ASEs is to transfer information between AR endpoints. This information transfer occurs over the conveyance paths of an AR. A conveyance path represents a one-way communication path used by an endpoint for input or output.

To support the role of the application process, the endpoint is configured with either one or two conveyance paths. Endpoints that only send, or only receive, are configured with either a send or receive conveyance path, and those that do both are configured with both. ARs with a single conveyance path are called uni-directional, and those with two conveyance paths are called bi-directional.

Uni-directional ARs are capable of conveying service requests only. To convey service responses, a bi-directional AR is necessary. Therefore, uni-directional ARs support the transfer of unconfirmed services in one direction only, while bi-directional ARs support the transfer of unconfirmed and confirmed services initiated by only one endpoint, or by both endpoints.

6.1.3.1.2.5.3 Trigger policy

Trigger policy indicates when APDUs are transmitted by the data-link layer over the network.

The first type is referred to as user-triggered. User-triggered AREPs submit FAL APDUs to the data-link layer for transmission at its earliest opportunity.

The second type is referred to as network-scheduled. Network scheduled AREPs submit FAL APDUs to the data-link layer for transmission according to a schedule configured by management.

This network scheduling mechanism can be either cyclic or one-time.

Network-scheduled ARs may also convey requests and responses asynchronously if the underlying layer has available bandwidth through use of the compel service. These transfers occur in addition to the scheduled transfers, and without being triggered from the network schedule.

6.1.3.1.2.5.4 Conveyance policy

Conveyance policy indicates whether APDUs are transferred according to a buffer model or a queue model. These models describe the method of conveying APDUs from sender to receiver.

Buffered ARs contain conveyance paths that have a single buffer at each endpoint. Updates to the source buffer are conveyed to the destination buffer according to the trigger policy of the AR. Updates to either buffer replace its contents with new data. In buffered ARs, unconveyed or undelivered data that has been replaced is lost. Additionally, data contained in a buffer may be read multiple times without destroying its contents.

Queued ARs contain conveyance paths that are represented as a queue between endpoints. Queued ARs convey data using a FIFO queue. Queued ARs are not overwritten; new entries are queued until they can be conveyed and delivered.

If a queue is full, a new message will not be placed into the queue.

NOTE The AR conveyance services are described abstractly in such a way that they are capable of being implemented to operate using buffers or queues. These services may be implemented in a number of ways. For example, they may be implemented such that the capability is provided to load the buffer/queue, and subsequently post it for transfer by the underlying data-link layer. Or, these services may be implemented such that these capabilities are combined so that the buffer/queue may be loaded and transferred in a single request. On the receiving side, these services may be implemented by delivering the data when it is received, or by indicating its receipt and allowing the user to retrieve it in a separate operation. Another option is to require the user to detect that the buffer or queue has been updated.

6.1.3.1.3 Underlying communications services

6.1.3.1.3.1 General

The AR ASE conveys FAL APDUs using the capabilities of the underlying data-link layer. Several characteristics are used to describe these capabilities. This subclause provides a description of each. These characteristics are specific to the data-link mapping defined in IEC 61158-6-8. Their precise specification can be found there.

6.1.3.1.3.2 Connection-oriented services

The underlying layer may support AR endpoints by providing connection-oriented services. Endpoints configured to operate over connection-oriented services may be capable of opening and closing connections if the connections they use are not pre-configured to be open.

6.1.3.1.3.3 Buffered and queued services

The underlying layer may support AR endpoints by providing buffered or queued services. These services can be used to implement buffers or queues required by certain classes of endpoints.

6.1.3.1.3.4 Cyclic and acyclic transfers

The underlying layer may support AR endpoints by providing cyclic or acyclic services.

The underlying layer may also provide for the acyclic transfer of APDUs concurrently with cyclic transfer of APDUs between the same endpoints.

6.1.3.1.4 AR establishment

For an AR endpoint to be used by an application process, the corresponding AR has to be active. When an AR is activated, it is referred to as "established".

AR establishment can occur in one of two ways. First, ARs can be pre-established. Pre-established means that the AE that maintains the endpoint context is created when the AP is connected to the network. In this case, communications among the applications involved in the AR may take place without first having to explicitly establish the AR. Any AR may be defined to be pre-established.

Second, ARs can be pre-defined, but not pre-established. Pre-defined means that the characteristics of the AR are known to each endpoint, but that their contexts have not been synchronized for operation. In this case, the use of the FAL Establish service is required to synchronize them and open them for data transfer.

6.1.3.1.5 Application relationship classes

AREPs are defined with a combination of characteristics to form different classes of ARs.

6.1.3.2 Application relationship endpoint class specifications

6.1.3.2.1 Formal model

The AR endpoint formal model defines the characteristics common to all AR endpoints. This class is not capable of being instantiated. It is present only for the inheritance of its attributes and services by its subclasses, each specified in a separate subclause of this standard.

All AR endpoint attributes are accessible through system Management and not through the Object Management ASE services defined in this standard.

FAL ASE:	AR ASE
CLASS:	AR ENDPOINT
CLASS ID:	32
PARENT CLASS:	TOP
SYSTEM MANAGEMENT ATTRIBUTES:	
1 (m) Attribute:	Local AP
2 (m) Attribute:	FAL Revision
3 (m) Attribute:	Dedicated (TRUE, FALSE)
4 (o) Attribute:	Transfer Syntax
SERVICES:	
1 (o) OpService:	AR-Unconfirmed Send
3 (o) OpService:	AR-Establish
5 (o) OpService:	AR-Abort

6.1.3.2.2 System management attributes

Local AP

This attribute identifies the AP attached or configured to use the AREP using a local reference.

FAL revision

This specifies the revision level of the FAL protocol used by this endpoint. The revision level is in the AR header of all FAL-PDUs transmitted.

Dedicated

The attribute specifies, when TRUE, that the endpoint is dedicated. When TRUE, the services of the AR ASE are accessed directly by the FAL User.

Transfer syntax

This optional attribute identifies the encoding rules to be used on the AR. When not present, the default FAL Transfer Syntax of this standard is used.

6.1.3.2.3 Services

All services defined for this class are optional. When an instance of the class is defined, at least one has to be selected.

AR-unconfirmed send

This optional service is used to send an unconfirmed service.

AR-establish

This optional service is used to establish (open) an AR.

AR-abort

This optional service is used to abort (abruptly terminate) an AR.

6.1.3.3 Application relationship ASE service specifications

6.1.3.3.1 Supported services

This subclause contains the definition of services that are unique to this ASE. The services defined for this ASE are

AR-Unconfirmed Send

AR-Establish

AR-Abort

The service, AR-Establish, contains the FAL PDU Body as part of the Result parameter in the response and confirmation primitives. The FAL PDU Body may contain either the positive or negative responses returned by the FAL User transparently to the AR ASE. Therefore, these services have a single Result parameter instead of separate Result(+) and Result(-) parameters that are commonly used to convey the positive and negative responses returned by the FAL User.

6.1.3.3.2 AR-unconfirmed send service

6.1.3.3.2.1 Service overview

This service is used to send AR-Unconfirmed request APDUs for FAL APO ASEs. The AR-Unconfirmed Send service may be requested at either endpoint of a one-to-one bi-directional AR, at the server endpoint of a one-to-one uni-directional AR.

6.1.3.3.2.2 Service primitives

The service parameters for this service are shown in Table 9.

Table 9 – AR-unconfirmed send

Parameter name	Req	Ind
Argument		
AREP	M	M
FAL Service Type	M	M (=)
FAL APDU Body	M	M (=)

Argument

The argument contains the parameters of the service request.

FAL service type

This parameter specifies the type of the service being conveyed.

FAL APDU body

This parameter specifies the service dependent body for the APDU.

6.1.3.3.2.3 Service procedure

The AR-Unconfirmed Send Service is a service that operates through a queue or buffer.

The requesting FAL ASE submits an AR-Unconfirmed send.request primitive to its AR ASE. The AR ASE builds an AR-Unconfirmed Send request APDU.

If the AREP is queued, the AR ASE queues the APDU for submission to the lower layer. If the AR is buffered, the AR ASE replaces the previous contents of the buffer with the APDU contained in the service primitive.

If the AREP is user-triggered, the AR ASE immediately requests the lower layer to transfer the APDU. If the AR is network-scheduled, the AR ASE requests the DL to transfer the data at the scheduled time. The data-link mapping indicates how the AR ASE coordinates its requests to transmit the data with the data-link layer.

NOTE The transmission schedule is managed by the underlying layer, not the AR-ASE. Refer to IEC 61158-3-8 and IEC 61158-4-8 for further details.

Upon receipt of the AR-Unconfirmed Send request APDU, the receiving AR ASE delivers an AR-Unconfirmed send.indication primitive to the appropriate FAL ASE as indicated by the FAL Service Type Parameter.

6.1.3.3.3 AR-establish service

6.1.3.3.3.1 Service overview

This confirmed service operates in a pair-wise manner between two AR endpoints to synchronize their contexts and activate them for the transfer of APDUs.

The endpoint context may be created during establishment or prior to establishment through System Management.

6.1.3.3.3.2 Service primitives

The service parameters for this service are shown in Table 10.

Table 10 – AR-establish service

Parameter name	Req	Ind	Rsp	Cnf
Argument				
AREP	M	M		
User Data	U	U (=)		
Result				
User Data			U	U (=)
NOTE The method by which a confirm primitive is correlated with its corresponding preceding request primitive is a local matter. See 1.2.				

Argument

The argument contains the parameters of the service request.

AREP

This parameter specifies sufficient information to locally identify the AREP to be established.

User data

This optional parameter specifies user supplied data that is to be conveyed with the service request.

Result

This parameter indicates that the service request succeeded or failed.

AREP

This parameter specifies sufficient information to locally identify the AREP to be established.

User data

This optional parameter specifies user supplied data that is to be conveyed with the service response.

6.1.3.3.3.3 Service procedure**6.1.3.3.3.3.1 1-to-1 AR establishment**

When used in support of 1-to-1 AREPs, the AR-Establish service causes Establish PDUs to be exchanged between calling and called DLSAPs as shown in Figure 14.

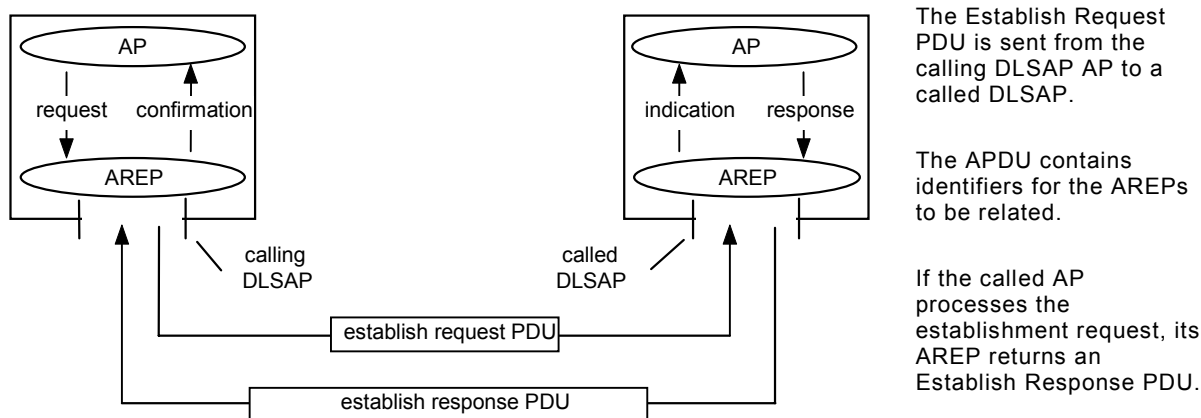


Figure 14 – 1-to-1 AR establishment

Upon receipt of an AR-Establish request service primitive, the calling FAL issues an Establish Request APDU to the called AP ASE that handles establishment of an AR. Upon receipt of an Establish indication primitive, the called AP ASE examines the parameters specified in it and returns the appropriate response in the AR-Establish response primitive.

6.1.3.3.3.2 Compatible AREP classes

Table 11 provides possible combinations of the AREP classes which may be related with the AR-Establish service. In this table the SERVER column contains a "-" to indicate that Server AREPs do not initiate AR establishment.

Table 11 – Valid combinations of AREP classes to be related

		Calling AREP Role		
		PEER	CLIENT	SERVER
Called AREP Role	PEER	YES	YES	--
	CLIENT	NO	NO	--
	SERVER	YES	YES	--

If the called AP decides to establish an AR, it issues an AR-Establish response primitive. The FAL returns the AR parameter which is used to refer to the AR being formed from the called service user. If the AR being established uses the Connection Oriented data-link layer and its explicit establishment is required, it is established before an Establish Response APDU is returned. The called AP then returns an Establish Response APDU with the Result(+) parameter to the calling AR_ASE. The calling AR_ASE issues an AR-Establish confirmation primitive in which the AR parameter is specified.

6.1.3.3.3.3 Conflict resolution

The normal establishment of ARs follows the general time-sequence for confirmed services. In the cases where each endpoint of an AR concurrently issues an AR-Establish request, a conflict arises. The algorithms for resolving conflicts of this type are specified in detail in IEC 61158-6-8.

6.1.3.3.4 AR-abort service

6.1.3.3.4.1 Service overview

The service is used by the FAL User to abruptly terminate an AR. It is always successful; the receiver of an Abort request or Abort request APDU always aborts the AR. This service may be used on any open AREP, whether or not the AR was pre-established or dynamically established using the establish service.

The AR-Abort Service is used to instruct the AR ASE to abruptly terminate all activity on an AREP and place it in the Closed state. Receipt of an AR-Abort request primitive causes the AR ASE to immediately close the AREP context and issue an Abort Request APDU to the remote AREPs. Receipt of an Abort Request APDU causes the AR ASE to immediately close the AR AREP context and deliver an AR-Abort request indication primitive to the user. The immediate close of each endpoint context causes all outstanding service requests to be cleared. All subsequent service primitives and APDUs received by the AR ASE for the aborted AR are discarded except for those of the AR-Establish service.

The AR-Abort service may be requested at either AREP of a one-to-one relationship.

The AR ASE may also initiate the abort service when it detects unrecoverable communication failures. In this case, the AR ASE delivers an AR-Abort indication primitive informing the user of the failure and closes the endpoint context.

6.1.3.3.4.2 Service primitives

The service parameters for this service are shown in Table 12.

Table 12 – AR-abort

Parameter name	Req	Ind
Argument		
AREP	M	M
Locally Generated		M
Originator	M	M (=)
Reason Code	M	M (=)
Additional Detail	U	U (=)

Argument

This parameter carries the information associated with the AR-Abort service.

Locally generated

This parameter specifies if the abort was locally generated, or not.

Originator

This parameter identifies the originator for the abort. Its valid values are DLL, FAL, or FAL-USER. The value DLL cannot be used in the request primitive.

Reason code

This parameter indicates the reason for the Abort. It may be supplied by either the provider or the user. One reason is defined: AR ASE error. Other reason code values may be supplied by the data-link layer, or by the user.

Additional detail

This optional parameter specifies user data that accompanies the indication. When used, the value submitted in the request primitive is delivered unchanged in the indication primitive.

6.1.3.3.4.3 Service procedure

The abort service is a service that operates through a queue or buffer.

If the user wishes to abort an AR, it submits an abort request primitive to its FAL AR ASE. If an AR ASE detects an unrecoverable local failure or a communication failure, it delivers an abort indication primitive to the endpoint user.

The FAL AR ASE builds an abort request APDU and conveys it on the specified AR if it is capable of doing so. It also transitions the AREP state to CLOSED

Upon receipt of the Abort Request APDU, each remote FAL AR ASE transitions its AREP to CLOSED and delivers an abort indication primitive to its user.

6.1.3.3.5 AR-data-send-acknowledge service

This service is defined for the FAL to provide the FAL user a service for transparent data transfer. This service can be used to exchange arbitrary data over a DLL.

For this service no specific ARPM state machine is available. The data flow is controlled by the FAL user.

Table 13 – AR-Data-Send-Acknowledge service parameters

Parameter name	Req	Ind	Cnf
Argument			
Destination Address	M	M (=)	
Source Address	M	M (=)	
Destination Node	M	M (=)	
Source Node	M	M (=)	
Destination Subnode	M	M (=)	
Source Subnode	M	M (=)	
Protocol Code	M	M (=)	
Block Number	M	M (=)	
Data	M	M (=)	
Result(+)			S
Result(-)			S
Error Info			M
NOTE The method by which a confirm primitive is correlated with its corresponding preceding request primitive is a local matter. See 1.2.			

Argument

The argument contains the parameters of the service request.

Destination address

This parameter specifies the logical destination address of the requested device. The broadcast address is FF FF FF.

Source address

This parameter specifies the logical source address of the local device.

Destination node

This parameter specifies the destination node of the requested device.

Source node

This parameter specifies the destination node of the local device.

Destination subnode

This parameter specifies the destination subnode of the requested device.

Source subnode

This parameter specifies the destination subnode of the local device.

Protocol code

This parameter specifies the FAL user protocol.

Block number

This parameter is the number of the PDU segment.

Data

This parameter specifies the PDU that has to be exchanged.

6.1.4 Variable ASE

6.1.4.1 Overview

In the Type 8 environment, application processes contain data that remote applications are able to read and write. The variable ASE defines the network visible attributes of application data and provides a set of services used to read, write, and report their values.

Individual variables are used to specify access to application data of any FAL-defined or user-defined data type. Services defined to access individual variables can be used to access one entire variables or one entry (per variable) in an array or one field (per variable) in a structure.

The formal model of the variable model is presented next, followed by a description of its services. IEC 61158-6-8 describes the abstract syntax and procedures for its protocol.

6.1.4.2 Requirements for access to variables

The structure of a variable value is defined by its data type. A variable's data type may be of any valid standardized or user defined data type. Standardized types and the facilities for defining user types are specified by the Data type Model.

6.1.4.3 Variable model class specification

6.1.4.3.1 Simple variable class specification

6.1.4.3.1.1 Simple variable formal model

FAL ASE:		VARIABLE ASE	
CLASS:		SIMPLE VARIABLE	
CLASS ID:		7	
PARENT CLASS:		TOP	
ATTRIBUTES:			
2	(m)	Attribute:	Data type
3	(m)	Attribute:	Length
5	(m)	Attribute:	Access Privilege
5.1	(m)	Attribute:	Password
5.2	(m)	Attribute:	Access Groups
5.3	(m)	Attribute:	Access Rights
6	(m)	Attribute:	Local Detail
SERVICES:			
1	(o)	OpsService:	Read
2	(o)	OpsService:	Write
5	(o)	OpsService:	Information Report

6.1.4.3.1.2 Attributes

Data type

This attribute is the numeric identifier of a FIXED-LENGTH or STRING data type.

Length

This attribute indicates the length of simple variables data types. For variables with STRING data types, this attribute is used to define the maximum length of the variable in octets. For variables with the FIXED-LENGTH data type, this attribute is used to reflect the length of the variable as specified by the data type.

Access privilege

This attribute specifies the access controls defined for this variable. It is composed of the following:

Password

This attribute specifies the password for the access rights. Its value is null if it is not used.

Access groups

This attribute identifies which of the eight user-defined access groups are defined for the variable. More than one access group may be defined.

Access rights

This attribute defines the type of access defined for the variable. Valid values are

- Right to Write for Access Groups
- Right to Read for Access Groups
- Right to Write for the registered Password
- Right to Read for the registered Password
- Right to Write for all Communication Partners
- Right to Read for all Communication Partners.

Local detail

This attribute specifies local information.

6.1.4.3.1.3 Services

All services defined for this class are optional. When an instance of the class is defined, at least one has to be supported.

Read

This optional service may be used to read a single variable object or variable list object. This service may be used in both the client/server model.

Write

This optional service may be used to update a single variable object or variable list object. This service may be used only in the client/server model.

Information report

This optional service is an unconfirmed service that may be used to report the value of a variable or variable list object. This service may be used in both the client/server model and the publisher/subscriber push model.

6.1.4.3.2 Array variable class specification

6.1.4.3.2.1 Formal model

FAL ASE:		VARIABLE ASE	
CLASS:		ARRAY VARIABLE	
CLASS ID:		8	
PARENT CLASS:		TOP	
ATTRIBUTES:			
2	(m)	Attribute:	Data type
2.1	(m)	Attribute:	Data type ID
3	(c)	Constraint:	Data type Format = FIXED-LENGTH STRING
3.1	(o)	Attribute:	Element Length
5	(o)	Attribute:	Number of Elements
6	(m)	Attribute:	Access Privilege
6.1	(m)	Attribute:	Password
6.2	(m)	Attribute:	Access Groups

6.3 (m) Attribute: Access Rights

7 (m) Attribute: Local Detail

SERVICES:

1 (o) OpsService: Read

2 (o) OpsService: Write

5 (o) OpsService: Information Report

6.1.4.3.2.2 Attributes

Data type

This attribute specifies the data type for elements of the array.

Element length

This conditional attribute indicates the length of an array element with an ARRAY, FIXED-LENGTH or STRING data type. For array elements with STRING data types, this attribute is used to define the length of an array element in octets. For variables with an ARRAY or FIXED-LENGTH data type, this attribute is used to reflect the length of an array element as specified by the data type.

Number of elements

This attribute specifies the number of array elements for variable objects that are defined as arrays. Array variables may be defined in one of two ways. Each of the ways, and the use of this attribute for each are shown below.

Method	Attribute Use
Reference to a data type with the format of ARRAY:	Reflects the number of elements defined for the array data type
Reference to a data type with the format of FIXED-LENGTH or STRING	Specifies the number of elements defined for the array variable

Access privilege

See the description of this attribute and its components defined in the Simple Variable class above.

Local detail

This attribute specifies local information.

6.1.4.3.2.3 Services

See the description of the services defined in the Simple Variable class above.

6.1.4.3.3 Record variable class specification

6.1.4.3.3.1 Formal model

FAL ASE:	VARIABLE ASE
CLASS:	RECORD VARIABLE
CLASS ID:	9
PARENT CLASS:	TOP
ATTRIBUTES:	
2 (m) Attribute:	Data type
3 (m) Attribute:	Access Privilege
3.1 (m) Attribute:	Password
3.2 (m) Attribute:	Access Groups
3.3 (m) Attribute:	Access Rights
4 (m) Attribute:	List of Local Detail
SERVICES:	

- | | | | |
|---|-----|-------------|--------------------|
| 1 | (o) | OpsService: | Read |
| 2 | (o) | OpsService: | Write |
| 5 | (o) | OpsService: | Information Report |

6.1.4.3.3.2 Attributes

Data type

This attribute specifies the data type for elements of the array.

Access privilege

See the description of this attribute defined for the Simple Variable class above.

Password

See the description of this attribute defined for the Simple Variable class above.

Access groups

See the description of this attribute defined for the Simple Variable class above.

Access rights

See the description of this attribute defined for the Simple Variable class above.

List of local detail

This attribute specifies the local detail for each element (field) of the record.

6.1.4.3.3.3 Services

See the description of the services defined for the Simple Variable class above.

6.1.4.4 Variable ASE service specification

6.1.4.4.1 Supported services

This subclause contains the definition of services that are unique to this ASE. The services defined for this ASE are

Read

Write

Information Report

The exact nature of the operation of variable services is determined, in part, by the application relationship over which they operate.

6.1.4.4.2 Read service

6.1.4.4.2.1 Service overview

This confirmed service may be used to read the value of a variable object. It may be used with application relationships configured to support the client/server model.

6.1.4.4.2.2 Service primitives

The service parameters for this service are shown in Table 14.

Table 14 – Read service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument				
AREP	M	M		
Invoke ID	U			
Variable Specifier	M	M (=)		
Numeric Identifier	S	S (=)		
Name	S	S (=)		
Result(+)			S	S (=)
Invoke ID				U (=)
Value			M	M (=)
Result(-)			S	S (=)
Invoke ID				U (=)
Error Info			M	M (=)
NOTE The method by which a confirm primitive is correlated with its corresponding preceding request primitive is a local matter. See 1.2.				

Argument

This parameter carries the parameters of the service invocation.

Variable specifier

This selector type parameter specifies the variable, variable list, or an element of an array or record variable.

Numeric identifier

This parameter identifies the variable by its Numeric Identifier (Index and Subindex).

Name

This parameter identifies the variable by its Name

Result(+)

This selection type parameter indicates that the service request succeeded.

Value

This parameter specifies the value read. For each of the variables, this parameter specifies the value of the variable. For variable lists, this parameter specifies the values of each of the variables in the list concatenated together in the order that they appear in the list. If any of the variables in a variable list could not be read, the service fails.

Result(-)

This selection type parameter indicates that the service request failed.

6.1.4.4.2.3 Service procedure

The Confirmed Service Procedure specified in 4.6 applies to this service.

6.1.4.4.3 Write service

6.1.4.4.3.1 Service overview

This confirmed service is used to write the value of a variable.

6.1.4.4.3.2 Service primitives

The service parameters for this service are shown in Table 15.

Table 15 – Write service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument				
AREP	M	M		
Invoke ID	U			
Variable Specifier	M	M (=)		
Numeric Identifier	S	S (=)		
Name	S	S (=)		
Value	M	M (=)		
Result(+)			S	S (=)
Invoke ID				U (=)
Result(-)			S	S (=)
Invoke ID				U (=)
Error Info			M	M (=)
NOTE The method by which a confirm primitive is correlated with its corresponding preceding request primitive is a local matter. See 1.2.				

Argument

The argument contains the parameters of the service request.

Variable specifier

This parameter specifies the variable, or an element of an array or record variable.

Numeric identifier

This parameter identifies the variable by its Numeric Identifier (Index and Subindex).

Name

This parameter identifies the variable by its Name.

Value

This parameter specifies the value to write. For variables, this parameter specifies the value of the variable. For variable lists, this parameter specifies the values of each of the variables in the list concatenated together in the order that they appear in the list.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

6.1.4.4.3.3 Service procedure

The Confirmed Service Procedure specified in 4.6 applies to this service.

6.1.4.4.4 Information report service

6.1.4.4.4.1 Service overview

This unconfirmed service is used by an application process to report the value of a variable to receiver(s) designated by the AR.

6.1.4.4.4.2 Service primitives

The service parameters for this service are shown in Table 16.

Table 16 – Information report service

Parameter name	Req	Ind
Argument		
AREP	M	M
Variable Specifier	C	C (=)
Numeric Identifier	S	S (=)
Name	S	S (=)
Value	M	M (=)

Argument

This parameter carries the parameters of the service invocation.

Variable specifier

This selector type parameter specifies the variable, or an element of an array or record variable.

Numeric identifier

This parameter identifies the variable by its Numeric Identifier (Index and Subindex).

Name

This parameter identifies the variable by its Name.

Value

This parameter specifies the value. For variables, this parameter specifies the value of the variable. For variable lists, this parameter specifies the values of each of the variables in the list concatenated together in the order that they appear in the list.

6.1.4.4.4.3 Service procedure

The Unconfirmed Service Procedure specified in 4.6 applies to this service.

6.1.5 Function invocation ASE

6.1.5.1 Overview

The FAL function invocation model defines two objects, the stateless action object and the state-oriented function invocation object.

Stateless function invocations, referred to as actions, run to completion when invoked and cannot be interrupted. In addition, some atomic actions return a value in response to being

invoked, while others do not. Those that do may be used to model software *functions*, and those that do not may be used to model software *procedures*.

State-oriented function invocations, on the other hand, may be controlled during their execution. Services are defined to suspend, resume, or abort them once they have been invoked. They do not return a value in response to being started. If it is necessary to abort a function invocation once it has been started, then it is defined as state-oriented.

State-oriented function invocations represent the network view of a specific invocation of a user function. If the user function can be invoked more than once simultaneously, then separate function invocation objects are needed to represent each invocation.

State-oriented function invocations may be used to model software processes or user operations that may be started and controlled.

NOTE An example of a state-oriented function invocation that may be used to model a user operation is the "playback" operation of a video recorder. Once it has been started, the playback operation may be stopped (paused) and later resumed.

To support the concept of software process modeling, the definition of state-oriented functional invocations includes optional attributes that can be used to relate them to load region objects.

The remainder of this subclause specifies the class definitions and services for the state-oriented *function invocation* object and the stateless *action* object.

6.1.5.2 Function invocation model class specifications

6.1.5.2.1 Function invocation class specification

6.1.5.2.1.1 Class overview

The function invocation class models the state-oriented function invocation. It may be used to model software processes or user functions whose operation may be controlled.

6.1.5.2.1.2 Formal model

FAL ASE:		FUNCTION INVOCATION ASE	
CLASS:		FUNCTION INVOCATION	
CLASS ID:		3	
PARENT CLASS:		TOP	
ATTRIBUTES:			
1	(m)	Attribute:	Access Privilege
1.1	(m)	Attribute:	Password
1.2	(m)	Attribute:	Access Groups
1.3	(m)	Attribute:	Access Rights
2	(m)	Attribute:	Deletable (TRUE,FALSE)
3	(m)	Attribute:	Reusable (TRUE,FALSE) Default : TRUE
4	(m)	Attribute:	Function Invocation State
54	(m)	Attribute:	Number of Related Objects In Use
SERVICES:			
1	(o)	OpsService:	Start
2	(o)	OpsService:	Stop
3	(o)	OpsService:	Resume
4	(o)	OpsService:	Reset

6.1.5.2.1.3 Attributes

Access Privilege

This attribute specifies the access controls defined for this function invocation. It is composed of the following:

Password

This attribute specifies the password for the access rights. Its value is null if it is not used.

Access groups

This attribute identifies which of the eight user-defined access groups are defined for the function invocation. More than one access group may be defined.

Access rights

This attribute defines the type of access defined for the function invocation. Valid values are

- Right to Delete for Access Groups(see note)
- Right to Start for Access Groups
- Right to Stop for Access Groups
- Right to Resume for Access Groups
- Right to Reset for Access Groups
- Right to Start for the Registered Password
- Right to Stop for the Registered Password
- Right to Resume for the Registered Password
- Right to Reset for the Registered Password
- Right to Start for all Communication Partners
- Right to Stop for all Communication Partners
- Right to Resume for all Communication Partners
- Right to Reset for all Communication Partners.

NOTE The right to delete has no meaning if the function invocation object is not deletable. This right requires the use of the Object Management Delete service, and not an operational service of the Function Invocation class.

Deletable

This attribute indicates, when TRUE, that the function invocation can be deleted using the Delete service. The value of this attribute is always TRUE for objects dynamically created with the Object Management ASE Create Service.

Reusable

This attribute indicates, when TRUE, that the function invocation may be executed more than once.

Function invocation state

This attribute indicates the current state of the function invocation. An enumerated set of values has been defined, and may be extended for local use within the AP to describe transient states of the Function Invocation. These transient state values for this attribute, however, are not visible over the network. The transitions between states and the events that cause transitions are user-defined.

UNRUNNABLE This state indicates that the function invocation is not executing and may not be executed.

IDLE This state indicates that the function invocation is not executing, but is capable of being executed.

RUNNING This state indicates that the function invocation is executing.

STOPPED This state indicates that the execution of a function invocation has been suspended.

STARTING This transient state indicates that the function invocation is being prepared for execution. Function invocations in this state are "ready to run".

STOPPING This transient state indicates that the execution of the function invocation is in the process of being stopped and its context saved.

RESUMING This transient state indicates that the function invocation is in the process of being returned to the executing state from a previously saved context.

RESETTING This transient state indicates that the function invocation is being reset to its initial context and removed from execution.

Number of related objects in use

This attribute indicates the number of related Load Region objects defined for this function invocation that are currently in the IN USE state. This attribute is incremented/decremented each time a related Load Region Object State Attribute goes/leaves the IN USE state. The Load Region State Machine gives the constraints for these transitions.

6.1.5.2.1.4 Services

Start

This optional service is used to request the function invocation to start executing. If this service is not present, it is assumed that the function invocation will be started using local means.

Stop

This optional service is used to request the function invocation to stop executing.

Resume

This optional service is used to request the function invocation to continue its execution.

Reset

This optional service is used to request the function invocation to return to its initial context and state.

6.1.5.2.2 Start service

6.1.5.2.2.1 Service overview

This confirmed service is used to request that a function invocation be started.

6.1.5.2.2.2 Service primitives

The service parameters for this service are shown in Table 17.

Table 17 – Start service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument				
AREP	M	M		
Invoke ID	U			
Key Attribute	M	M (=)		
Numeric Identifier	S	S (=)		
Name	S	S (=)		
Result(+)			S	S (=)
Invoke ID				U (=)
Result(-)			S	S (=)
Invoke ID				U (=)
Error Info			M	M (=)
Function Invocation State			C	C (=)
NOTE The method by which a confirm primitive is correlated with its corresponding preceding request primitive is a local matter. See 1.2.				

Argument

This parameter carries the parameters of the service invocation.

Key attribute

This parameter specifies one of the key attributes of the function invocation object.

Numeric identifier

This parameter identifies the function invocation object by its Numeric Identifier (Index).

Name

This parameter identifies the function invocation object by its Name.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

Function invocation state

This conditional parameter specifies the current state of the function invocation. It is present if the error code indicates an object state conflict.

6.1.5.2.2.3 Service procedure

The Confirmed Service Procedure specified in 4.6 applies to this service.

6.1.5.2.3 Stop service

6.1.5.2.3.1 Service overview

This confirmed service is used to stop a function invocation retaining its context so that it may be resumed.

6.1.5.2.3.2 Service primitives

The service parameters for this service are shown in Table 18.

Table 18 – Stop service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument				
AREP	M	M		
Invoke ID	U			
Key Attribute	M	M (=)		
Numeric Identifier	S	S (=)		
Name	S	S (=)		
Result(+)			S	S (=)
Invoke ID				U (=)
Result(-)			S	S (=)
Invoke ID				U (=)
Error Info			M	M (=)
Function Invocation State			C	C (=)
NOTE The method by which a confirm primitive is correlated with its corresponding preceding request primitive is a local matter. See 1.2.				

Argument

This parameter carries the parameters of the service invocation.

Key attribute

This parameter specifies one of the key attributes of the function invocation object.

Numeric identifier

This parameter identifies the function invocation object by its Numeric Identifier (Index).

Name

This parameter identifies the function invocation object by its Name.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

Function invocation state

This conditional parameter specifies the current state of the function invocation. It is present if the error code indicates an object state conflict.

6.1.5.2.3.3 Service procedure

The Confirmed Service Procedure specified in 4.6 applies to this service.

6.1.5.2.4 Resume service

6.1.5.2.4.1 Service overview

This confirmed service is used to resume the execution of a function invocation that has been stopped. Execution is resumed using the context saved when the function invocation was stopped.

6.1.5.2.4.2 Service primitives

The service parameters for this service are shown in Table 19.

Table 19 – Resume service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument				
AREP	M	M		
Invoke ID	U			
Key Attribute	M	M (=)		
Numeric Identifier	S	S (=)		
Name	S	S (=)		
Result(+)			S	S (=)
Invoke ID				U (=)
Result(-)			S	S (=)
Invoke ID				U (=)
Error Info			M	M (=)
Function Invocation State			C	C (=)
NOTE The method by which a confirm primitive is correlated with its corresponding preceding request primitive is a local matter. See 1.2.				

Argument

This parameter carries the parameters of the service invocation.

Key attribute

This parameter specifies one of the key attributes of the function invocation object.

Numeric identifier

This parameter identifies the function invocation object by its Numeric Identifier (Index).

Name

This parameter identifies the function invocation object by its Name.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

Function invocation state

This conditional parameter specifies the current state of the function invocation. It is present if the error code indicates an object state conflict.

6.1.5.2.4.3 Service procedure

The Confirmed Service Procedure specified in 4.6 applies to this service.

6.1.5.2.5 Reset service

6.1.5.2.5.1 Service overview

This confirmed service is used to reset a function invocation with its initial context. Its initial context is defined as the context of the function invocation set by its initialization procedures.

6.1.5.2.5.2 Service primitives

The service parameters for this service are shown in Table 20.

Table 20 – Reset service parameters

Parameter name	Req	Ind	Rsp	Cnf
Argument				
AREP	M	M		
Invoke ID	U			
Key Attribute	M	M (=)		
Numeric Identifier	S	S (=)		
Name	S	S (=)		
Result(+)			S	S (=)
Invoke ID				U (=)
Result(-)			S	S (=)
Invoke ID				U (=)
Error Info			M	M (=)
Function Invocation State			C	C (=)
NOTE The method by which a confirm primitive is correlated with its corresponding preceding request primitive is a local matter. See 1.2.				

Argument

This parameter carries the parameters of the service invocation.

Invoke ID

This parameter identifies this invocation of the service. It is used to associate service requests with responses. Therefore, no two outstanding service invocations may be identified by the same Invoke ID value.

Key attribute

This parameter specifies one of the key attributes of the function invocation object.

Numeric identifier

This parameter identifies the function invocation object by its Numeric Identifier (Index).

Name

This parameter identifies the function invocation object by its Name.

Result(+)

This selection type parameter indicates that the service request succeeded.

Result(-)

This selection type parameter indicates that the service request failed.

Function invocation state

This conditional parameter specifies the current state of the function invocation. It is present if the error code indicates an object state conflict.

6.1.5.2.5.3 Service procedure

The Confirmed Service Procedure specified In Clause 4 applies to this service.

6.1.5.3 Function invocation state machine

Table 21 gives the State Transitions for a Function Invocation object. The state “NON EXISTENT” indicates that the object does not exist. It is not defined as a valid value for the function invocation state attribute because attributes are created and deleted with the object (and therefore have no value when the object does not exist).

Table 21 – State transitions for a function invocation object

Transition	Current State	Events	Next State
5	IDLE	Start.Ind	STARTING
6	STARTING	Start succeeds: e.g. Related Load Region Object in state LOADED or IN-USE For the Function Invocation object, update the attributes: Number of Related Object In Use List of Related Objects(if supported) For each related Load Region Object, update the attributes: Number of Related Objects In Use List of Related Objects(if supported) Start.Rsp(+)	RUNNING
7	STARTING	Start fails, non destructive: e.g. Related Load Region Object not in state LOADED or IN USE Start.Rsp(-)	IDLE
8	STARTING	Start fails, destructive Start.Rsp(-)	UN-RUNNABLE
9	RUNNING	Stop.Ind	STOPPING
10	STOPPING	Stop succeeds: e.g. Related Load Region Object in state IN USE For the Function Invocation object, update the attributes: Number of Related Object In Use List of Related Objects(if supported) For each related Load Region Object, update the attributes: Number of Related Objects In Use List of Related Objects(if supported) Stop.Rsp(+)	STOPPED
11	STOPPING	Stop Execution fails, non destructive Stop.Rsp(-)	RUNNING
12	STOPPING	Stop fails, destructive For the Function Invocation object, update the attributes: Number of Related Object In Use List of Related Objects(if supported) For each related Load Region Object, update the attributes: Number of Related Objects In Use List of Related Objects(if supported) Stop.Rsp(-)	UN-RUNNABLE
13	STOPPED	Resume.Ind	RESUMING
14	RESUMING	Resume succeeds: e.g. Related Load Region Object in state Loaded or In-Use For the Function Invocation object, update the attributes: Number of Related Object In Use List of Related Objects(if supported) For each related Load Region Object, update the attributes: Number of Related Objects In Use List of Related Objects(if supported) Resume.Rsp(+)	RUNNING
15	RESUMING	Resume fails, non destructive: e.g. Load Region not in state LOADED or IN-USE Resume.Rsp(-)	STOPPED

Transition	Current State	Events	Next State
16	RESUMING	Resume fails, destructive For each related Load Region Object, update the attributes: Number of Related Objects In Use List of Related Objects(if supported) Resume.Rsp(-)	UNRUNNABLE
17	STOPPED	Reset.Ind	RESETTING
18	RESETTING	Reset succeeds, reusable=TRUE Reset.Rsp(+)	IDLE
19	RESETTING	Reset succeeds, reusable = false Reset.Rsp(+)	UNRUNNABLE
20	RESETTING	Reset fails, non destructive Reset.Rsp(-)	STOPPED
21	RESETTING	Reset fails, destructive Reset.Rsp(-)	UNRUNNABLE
24	RUNNING	End of Program && Reusable=TRUE For the Function Invocation object, update the attributes: Number of Related Object In Use List of Related Objects(if supported) For each related Load Region Object, update the attributes: Number of Related Objects In Use List of Related Objects(if supported)	IDLE
25	RUNNING	End of Program && Reusable = false For the Function Invocation object, update the attributes: Number of Related Object In Use List of Related Objects(if supported) For each related Load Region Object, update the attributes: Number of Related Objects In Use List of Related Objects(if supported)	UNRUNNABLE
26	RUNNING	Program stopped For the Function Invocation object, update the attributes: Number of Related Object In Use List of Related Objects(if supported) For each related Load Region Object, update the attributes: Number of Related Objects In Use List of Related Objects(if supported)	STOPPED

6.2 ARs

6.2.1 Queued user-triggered bi-directional with flow control (QUB-FC) AR endpoint class specification

6.2.1.1 Class overview

This class is defined to support asynchronous operation of the client/server model using queues. It provides for the conveyance of confirmed and unconfirmed services using application layer flow control for unconfirmed services and uses connection-oriented data link services for the exchanges. The data link priority for the transfers is specified separately for each transfer.

The behavior of this class is described as follows.

An AR ASE user wishing to convey a request APDU submits it as an AR ASE Service Data Unit to its AREP. The AREP sending the request APDU queues it to its underlying layer for transfer at the next available opportunity with exception of unconfirmed acknowledged services. The latter will be ignored if the outstanding service counter has reached the negotiated maximum.

The AREP receiving the request APDU from its underlying layer, queues it for delivery to its AR ASE user, in the order that it was received. If the received APDU contains an unconfirmed acknowledged request, the AREP issues the appropriated indication to the AR ASE user and a UCA_AckPDU to the underlying layer for transfer.

For a confirmed service request the AREP receiving the request APDU accepts the corresponding response APDU from its AR ASE user and queues it to the underlying layer for transfer.

The AREP that issued the request APDU receives the response APDU from its underlying layer and queues it for delivery to its AR ASE user in the order that it was received and decrements the appropriate flow control counter.

The following summarizes the characteristics of this AREP class.

Roles	Client Server Peer
Cardinality	1-to-1

6.2.1.2 Formal model

FAL ASE:	AR ASE
CLASS:	Queued User-triggered Bi-directional with Flow Control AREP
CLASS ID:	46
PARENT CLASS:	AR Endpoint
NETWORK MANAGEMENT ATTRIBUTES:	
1	(m) Attribute: Role (CLIENT, SERVER, PEER)
2	(m) Attribute: AREP State
3	(m) Attribute: Remote Address Configuration Type (LINKED)
4	(m) Attribute: Maximum Outstanding Requests Calling
5	(m) Attribute: Maximum Outstanding Requests Called
6	(m) Attribute: Max Outstanding Unconfirmed Requests Client (maxUCSC)
7	(m) Attribute: Max Outstanding Unconfirmed Requests Server (maxUCSS)
8	(m) Attribute: Maximum Outstanding Requests Client (maxOSCC)
9	(m) Attribute: Maximum Outstanding Requests Server (maxOSCS)
10	(m) Attribute: Initiator (TRUE, FALSE)
11	(o) Attribute: Remote AP Name
12	(m) Attribute: Transmit DL Mapping Reference
13	(m) Attribute: Receive DL Mapping Reference
14	(m) Attribute: CIU
SERVICES:	
1	(o) OpService: Confirmed Send
2	(o) OpService: Unconfirmed Send
3	(o) OpService: Establish
4	(o) OpService: Abort

6.2.1.3 Network management attributes

Role

This attribute specifies the role of the AREP. The valid values are:

PEER (CLIENT/SERVER) Endpoints of this type may perform as either clients or servers, or both simultaneously. Endpoints of this type indicate whether or not they are to be the initiator of the AR establishment process.

CLIENT Endpoints of this type issue confirmed and unconfirmed service Request-APDUs to servers and receive confirmed service Response-APDUs.

SERVER Endpoints of this type receive confirmed and unconfirmed service Request-APDUs from clients and issue confirmed service Response-APDUs to them. They may also issue unconfirmed service Request APDUs to clients.

AREP state

This attribute specifies the state of the AREP. The values for this attribute are specified in IEC 61158-6-8.

Remote address configuration type

This attribute specifies how the remote address of the AREP is configured. The valid values are:

LINKED The value of "LINKED" indicates that the destination of the FAL-PDUs is configured with the RemoteDisapAddress attribute contained in the DL Mapping (specified in IEC 61158-6-8).

Max outstanding requests calling

This attribute indicates the maximum number of responses that the AREP may be expecting from the remote AREP.

Max outstanding requests called

This attribute indicates the maximum number of responses that the AR Endpoint may be expecting from its local user.

Max outstanding unconfirmed requests client (maxUCSC)

This attribute indicates the maximum number of requests that the AR Endpoint may be expecting from its local user.

Max outstanding unconfirmed requests server (maxUCSS)

This attribute indicates the maximum number of requests that the AREP may be expecting from the remote AREP.

Maximum outstanding requests client (maxOSCC)

This attribute indicates the maximum number of responses that the AREP may be expecting from the remote AREP.

Maximum outstanding requests server (maxOSCS)

This attribute indicates the maximum number of responses that the AR Endpoint may be expecting from its local user.

Initiator

This attribute indicates, when TRUE, that the endpoint has been configured to initiate the establishment of the AR. One and only one of the AREPs involved in an AR is the initiator. When this AREP is a client, the value of this attribute is always TRUE. When this AREP is a server, the value of this attribute is always FALSE. When this AREP is a Peer, the value of

this attribute may be either, as long as both AREPs in the AR are not configured to be the initiator.

Remote AP name

This optional attribute specifies the name of the AP attached at the remote AREP.

Transmit DL mapping reference

This attribute provides a reference to the underlying data-link layer mapping for the transmit conveyance path for this AREP. DL mappings for the data-link layer (IEC 61158-3-8) are specified in IEC 61158-6-8.

Receive DL mapping reference

This attribute provides a reference to the underlying data-link layer mapping for the receive conveyance path for this AREP. DL mappings for the data-link layer (IEC 61158-3-8) are specified in IEC 61158-6-8.

CIU

This attribute (CIU - Control Interval User-triggered) contains the control interval for monitoring of a user triggered connection. The attribute is set by network management and used by the ARPM. If this attribute specifies a value not 0 then the ARPM transmits an Idle PDU while AR ASE user has not passed any service during the control interval (CIU/3). If CIU expires then the connections will be terminated.

6.2.1.4 Services

Confirmed send

This optional service is used to send a confirmed service on an AR.

Establish

This service is used to establish an AR.

Abort

This service is used to disconnect an AR.

Unconfirmed send

This service is used to send an unconfirmed service on an AR.

6.2.2 Buffered network-scheduled uni-directional (BNU) AR endpoint class specification

6.2.2.1 Class overview

This class is defined to support the “push” model for scheduled, buffered distribution of unconfirmed services to one or more application processes.

The behavior of this type of AR can be described as follows.

An AR ASE user wishing to convey a request or response APDU submits it as an AR ASE Service Data Unit to its AREP for distribution. The AREP sending the request or response APDU writes it into the data-link layer buffer, completely replacing the existing contents of the buffer. The data-link layer transfers the buffer contents at the next scheduled transfer opportunity. The buffer is also transferred whenever a request to transmit is received, either locally or from the network.

NOTE Unscheduled requests to transmit have no effect on the behavior of this type of AR. They are treated as “out of band” with respect to the AR behavior.

If the AREP that sent the APDU receives another APDU before the buffer contents are transmitted, the buffer contents will be replaced with the new APDU, and the previous APDU

will be lost. When the buffer contents are transmitted, the AR ASE notifies the user of the transmission.

At the receiving endpoint, the APDU is received from the network and is immediately written into the buffer, completely overwriting the existing contents of the buffer. The endpoint notifies the user that the APDU has arrived and delivers it to the user according to the local user interface. If the APDU has not been delivered before the next APDU arrives, it will be overwritten by the next APDU and lost.

An FAL user receiving the buffered transmission may later request to receive the currently buffered APDU.

6.2.2.2 Formal model

FAL ASE:	AR ASE
CLASS:	Buffered Network-Scheduled Uni-directional AR Endpoint
CLASS ID:	38
PARENT CLASS:	AR Endpoint
NETWORK MANAGEMENT ATTRIBUTES:	
1 (m) Attribute:	Role (PUSH PUBLISHER, PUSH SUBSCRIBER)
2 (m) Attribute:	AREP State
2 (c) Constraint	Role == PUSH SUBSCRIBER
3 (m) Attribute:	DL Mapping Reference
SERVICES:	
1 (o) OpService:	Unconfirmed Send
3 (o) OpService:	Get Buffered Message

6.2.2.3 Network management attributes

Role

This attribute specifies the role of the AREP. The valid values are:

PUSH-PUBLISHER Endpoints of this type publish their data issuing unconfirmed service Request-APDUs.

PUSH-SUBSCRIBER Endpoints of this type receive data published in confirmed service Response-APDUs.

AREP state

This attribute specifies the state of the AREP. The values for this attribute are specified in IEC 61158-6-8.

DL mapping reference

For PUSH-PUBLISHER AREPs, this attribute specifies the mapping to the transmit conveyance path. For PUSH-SUBSCRIBER AREPs, this attribute specifies the mapping to the receive conveyance path. DL mapping attributes for the data-link layer (IEC 61158-3-8) are specified in IEC 61158-6-8.

6.2.2.4 Services

Unconfirmed send

This optional service is used to send an unconfirmed service on an AR.

Get buffered message

This local service is used to retrieve an APDU from the buffer used by an AR.

6.2.3 Queued user-triggered bi-directional transparent mode (QUB-TM) AR endpoint formal model

6.2.3.1 Description

This class is defined to support asynchronous operation of the publisher/subscriber model using queues. It provides a conveyance path for arbitrary data that are exchanged over a communication system.

The behavior of this class is described as follows.

An AR ASE user wishing to convey a request APDU submits it as an AR-Data-Send-Acknowledge Service to its AREP. The AREP sending the request APDU queues it to its underlying layer for transfer at the next available opportunity. The latter will be ignored if the outstanding service counter has reached the negotiated maximum.

The AREP receiving the request APDU from its underlying layer, queues it for delivery to its AR ASE user, in the order that it was received.

The following summarizes the characteristics of this AREP class.

Roles	PUSH PUBLISHER PUSH SUBSCRIBER
Cardinality	1-to-1

6.2.3.2 Formal class definition

FAL ASE:	AR ASE
CLASS:	Queued User-triggered Bi-directional Transparent Mode AREP
CLASS ID:	50
PARENT CLASS:	AR Endpoint
NETWORK MANAGEMENT ATTRIBUTES:	
1	(m) Attribute: Role (PUSH Publisher, PUSH Subscriber)
2	(m) Attribute: Maximum Outstanding Requests Calling
3	(m) Attribute: Maximum Outstanding Requests Called
4	(m) Attribute: Transmit DL Mapping Reference
5	(m) Attribute: Receive DL Mapping Reference
SERVICES:	
1	(m) OpsService: AR-Data-Send-Acknowledge

6.2.3.3 Network management attributes

6.2.3.3.1 Role

This attribute specifies the role of the AREP. The valid values are:

PUSH PUBLISHER Endpoints of this type publish their data issuing AR-Data-Acknowledge service Request-APDUs.

PUSH SUBSCRIBER Endpoints of this type receive data published in AR-Data-Acknowledge service Request-APDUs.

6.2.3.3.2 Max outstanding requests calling

This attribute indicates the maximum number of requests that the AREP may be expecting from the remote AREP.

6.2.3.3.3 Max outstanding requests called

This attribute indicates the maximum number of requests that the AR Endpoint may be expecting from its local user.

6.2.3.3.4 Transmit DL mapping reference

This attribute provides a reference to the underlying data-link layer mapping for the transmit conveyance path for this AREP. DL mappings for the data-link layer (IEC 61158-3-8) are specified in IEC 61158-6-8.

6.2.3.3.5 Receive DL mapping reference

This attribute provides a reference to the underlying data-link layer mapping for the receive conveyance path for this AREP. DL mappings for the data-link layer (IEC 61158-3-8) are specified in IEC 61158-6-8.

6.2.3.4 Services

6.2.3.4.1 AR-data-send-acknowledge

This service is used to exchange the PDU on an AR.

6.3 Summary of FAL classes

This subclause contains a summary of the defined FAL Classes. The Class ID values have been assigned to be compatible with existing standards. Table 22 provides a summary of the classes.

Table 22 – FAL class summary

FAL ASE	Class	Class ID
Application Process	Application Process	16
Data type	Fixed Length & String Data type	5
	Structure Data type	6
Application Relationship	AREP	32
	BNU	38
	QUB-FC	46
	QUB-TM	50
Variable	Fixed Length & String Variable	7
	Array Variable	8
	Data Structure Variable	9
Function Invocation	Function Invocation	3

6.4 Permitted FAL services by AREP role

Table 23 below defines the valid combinations of services and AREP roles (which service APDUs and AREP with the specified role can send or receive). The Unc and Cnf columns indicate whether the service listed in the left-hand column is unconfirmed (Unc) or confirmed (Cnf).

Table 23 – Services by AREP role

FAL Services	Unconfirmed	Confirmed	Client	Server	Push Publisher	Push Subscriber
			req rcv	req rcv	req rcv	req rcv
Mgt ASE						
Get Attributes		X	X	X		
AP ASE						
Identify		X	X	X		
Get Status		X	X	X		
Initiate		X	X	X		
Terminate		X	X	X		
Reject		X	X			
AR ASE						
AR-Unconfirmed Send					X	X
AR-Establish					X	X
AR-Abort					X	X X
AR-Data-Send-Acknowledged					X	X
Variable ASE						
Read		X	X	X		
Write		X	X	X		
Information Report	X		X	X		
Function Invocation ASE						
Start		X	X	X		
Stop		X	X	X		
Resume		X	X	X		
Reset		X	X	X		

Bibliography

IEC/TR 61158-1 (Ed.2.0), *Industrial communication networks – Fieldbus specifications – Part 1: Overview and guidance for the IEC 61158 and IEC 61784 series*

IEC 61158-2 (Ed.4.0), *Industrial communication networks – Fieldbus specifications – Part 2: Physical layer specification and service definition*

IEC 61158-3-8, *Industrial communication networks – Fieldbus specifications – Part 3-8: Data-link layer service definition – Type 8 elements*

IEC 61158-4-8, *Industrial communication networks – Fieldbus specifications – Part 4-8: Data-link layer protocol specification – Type 8 elements*

IEC 61158-6-8, *Industrial communication networks – Fieldbus specifications – Part 6-8: Application layer protocol specification – Type 8 elements*

IEC 61784-1 (Ed.2.0), *Industrial communication networks – Profiles – Part 1: Fieldbus profiles*

IEC 61784-2, *Industrial communication networks – Profiles – Part 2: Additional fieldbus profiles for real-time networks based on ISO/IEC 8802-3*

INTERNATIONAL
ELECTROTECHNICAL
COMMISSION

3, rue de Varembé
P.O. Box 131
CH-1211 Geneva 20
Switzerland

Tel: + 41 22 919 02 11
Fax: + 41 22 919 03 00
info@iec.ch
www.iec.ch